

AI-Based Virtual Helpdesk for University Department (IT dept)
Students

By

Muhammad Bilal Aqeel

(2022-GCUF-02661)

Abdul Waleed

(2022-GCUF-02684)

BACHELOR OF SCIENCE
IN
INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD.

June 2026

DECLARATION

The work reported in this documentation was carried out by us under the supervision of **Dr. Muhammad Younus** Department of Information Technology Government College University, Faisalabad, Pakistan.

We hereby declare that the title of project **AI-Based Virtual Helpdesk for University Department (IT dept) Students** and the contents of the documentation are the product of our own work and no part has been copied from any published source (except the references, standard algorithms /equations /formulas/protocols etc.) We further declare that this work has not been submitted for award of any other degree /diploma. The University may take action if the information provided is found inaccurate at any stage.

Signature of the Student

Muhammad Bilal Aqeel

2022-GCUF-02661

Signature of the Student

Abdul Waleed

2022-GCUF-02684

CERTIFICATE BY SUPERVISORY COMMITTEE

We certify that the contents and form of documentation submitted by Mr. Muhammad Bilal Aqeel Registration No: 2022-GCUF-02661 and by Mr. Abdul Waleed Registration No: 2022-GCUF-02684. Has been Found satisfactory and in accordance with the prescribed format. We recommend it to be processed for the evaluation by the External Examiner for the award of Degree.

Signature of Supervisor

Name.....

Designation with stamp.....

Member of Supervisory Committee

Signature

Name.....

Designation with stamp.....

Member of Supervisory Committee

Signature

Name.....

Designation with stamp.....

Chairperson

Signature

Name.....

Designation with stamp.....

DEDICATED

TO

We dedicate this Final Year Project to our beloved parents, whose unconditional love, prayers, sacrifices, and continuous encouragement have been the foundation of our success throughout our academic journey. Their unwavering support and belief in our abilities motivated us to overcome challenges and strive for excellence.

We also dedicate this work to our respected teachers and mentors, who shared their knowledge, guidance, and valuable insights, helping us develop both academically and professionally. Their dedication to education inspired us to pursue learning with commitment and determination.

Furthermore, we dedicate this project to all those who contributed directly or indirectly to our educational growth and personal development. Their support, encouragement, and trust played a significant role in the successful completion of this project. May this achievement serve as a reflection of our gratitude and appreciation for everyone who has been a part of our journey.

ACKNOWLEDGEMENTS

Our foremost gratitude and prayers belong to **ALMIGHTY ALLAH**, the most merciful and the most compassionate, whose divine guidance and blessings sustained us at every step of this academic pursuit. It is entirely by His grace and wisdom that we found the strength to complete this endeavor, especially during challenging times when He granted us the clarity to move forward. Along with this, we offer our deepest respect and devotion to His Holy Prophet '**MUHAMMAD**' (Peace be upon him), the most perfect and exalted creation ever to walk the earth, who remains an eternal source of enlightenment and knowledge for all mankind.

We extend our heartfelt appreciation to our esteemed supervisor and the dedicated faculty members of the Department of Information Technology. We are incredibly grateful for your invaluable mentorship, unwavering patience, and for guiding us through this rigorous academic journey without any judgment. By creating a supportive learning environment, you allowed us to learn from our mistakes, refine our skills, and push the boundaries of our capabilities.

We are also deeply indebted to our parents, the true foundation of our lives. We cannot thank you enough for your endless sacrifices, unconditional love, and constant encouragement. This milestone is a direct result of your prayers and the unwavering support system you have provided for us since the very beginning.

To our friends and cousins, thank you for being a constant source of strength and positivity. Your belief in our abilities even when we doubted ourselves motivated us to keep going whenever the workload felt overwhelming. Your laughter, support, and presence made this journey much easier to navigate.

Lastly, we take a moment to appreciate our own efforts as a team. Managing long hours, overcoming technical hurdles, and staying dedicated required immense mutual discipline and resilience. We are proud of our persistence, our ability to learn from setbacks, and the collaborative spirit that brought this project to a successful completion. This work stands as a symbol of our shared growth and hard work for the future.

Muhammad Bilal Aqeel

Abdul Waleed

Project Source Code & GitHub Repository

The complete and fully functional source code of this Final Year Project is publicly hosted on GitHub. The repository contains the entire implemented codebase of the AI-Based Virtual Helpdesk system, organized across four service directories: the React Native/Expo mobile application (frontend/), the Node.js/Express REST API backend (backend/server/), the Python FastAPI AI service (backend/ai-services/), and the React admin web panel (admin/). The repository reflects the final, submitted, and working state of the project as developed by the authors during the academic year 2026 at the Department of Information Technology, Government College University Faisalabad. All modules, controllers, services, models, and configuration files are version-controlled and accessible through the repository link and QR code provided below.

GitHub Repository Link

The following URL is the direct link to the public GitHub repository of this Final Year Project. The repository is hosted under the GitHub account **DevBilal17** and can be accessed from any web browser on a desktop or mobile device. Visiting this link will open the repository's main page on GitHub, where the complete project structure, all source files, and the full version history of the project can be explored. No login or authentication is required to view the repository, as it is publicly accessible.

[AI-BASED-VIRTUAL-HELPDESK-FOR-UNIVERSITY-DEPARTMENT-STUDENTS](#)

QR Code

The QR code presented below encodes the GitHub repository URL of this Final Year Project. This QR code is provided as a convenient alternative to the text-based URL for readers accessing this document in its printed form.



TABLE OF CONTENTS

Contents

Software Requirement Specification.....	1
1.1 Introduction.....	1
1.2 Project Objectives.....	2
1.2.1 General Objectives	2
1.2.2 Specific Objectives	2
1.3 Scope of the System.....	3
1.4 Feasibility Analysis	3
1.4.1 Technical Feasibility	3
1.4.2 Economic Feasibility	4
1.4.3 Operational Feasibility	5
1.5 Stakeholders	6
1.6 Users of the System	7
1.6.1 Students	7
1.6.2 Administrators	8
1.7 Functional Requirements	8
1.7.1 List of Functional Requirements	9
1.7.2 Description of each requirement in Requirement Shell	10
1.8 Non-Functional Requirements	25
1.9 Schedule of the Project	28
Project Analysis	33
2.1 Use Case Model.....	33
2.2 Description of Each Use Case in Fully Dressed Format	35
2.2.1 UC-01	35
2.2.2 UC-02	36

2.2.3 UC-03	37
2.2.4 UC-04	38
2.2.5 UC-05	39
2.2.6 UC-06	40
2.2.7 UC-07	41
2.2.8 UC-08	42
2.2.9 UC-09	44
2.3 System Sequence Diagrams	44
2.3.1 SSD — Natural Language Query Submission (UC-01)	45
2.3.2 SSD — Voice-Based Query Submission (UC-02)	45
2.3.3 SSD — User Registration and Login (UC-05)	46
2.3.4 SSD — Knowledge Document Upload and RAG Indexing (UC-06)	47
2.3.5 SSD — Location and QR Code Management (UC-08)	47
2.4 Domain Model	48
Project Design	50
3.1 Design Class Diagram	50
3.2 Entity Relationship Diagram (ERD)	53
3.3 System Architecture Diagram	55
3.4 Database Design Details	57
Implementation	59
4.1 Code/Algorithm of Main Scenarios and Main Units of the System	59
4.1.1 JWT Authentication Token Generation	59
4.1.2 User Database Schema (MongoDB)	60
4.1.3 User Login Controller	61
4.1.4 Chat Request Processing Controller	62
4.1.5 Multi-Turn Conversation Memory Retrieval	63

4.1.6 Query Normalization Algorithm	64
4.1.7 Intent Classification Algorithm	65
4.1.8 RAG Document Retrieval Algorithm	66
4.1.9 Prompt Construction for LLM	67
4.1.10 LLaMA Response Generation via Groq API	68
4.2 Tools and Technologies Used	69
4.3 System Modules Description	73
4.3.1 AI Chatbot Module (RAG Pipeline)	73
4.3.2 Voice Integration Module	75
4.3.3 Indoor Navigation Module	76
4.3.4 QR Code Module	77
4.3.5 Authentication and User Management Module	77
4.3.6 Knowledge Management Module (Admin Panel)	78
4.3.7 Chat History and Session Management Module	79
4.3.8 Admin Dashboard and Statistics Module	80
System Testing	81
5.1 Test Cases	81
5.2 Testing Types	91
5.2.1 Unit Testing	91
5.2.2 Integration Testing	93
5.2.3 System Testing	94
5.2.4 User Acceptance Testing (UAT)	95
5.3 Test Summary Report	96
User Interfaces	98
6.1 Admin Panel Interfaces	98

6.1.1 Authentication & Main Dashboard	98
6.1.2 User Management Module	99
6.1.3 Knowledge Base & Data Management	100
6.1.4 FAQ Management Module	101
6.2 Student Portal Interfaces	102
6.2.1 Student Authentication Framework	102
6.2.2 Student Home & Dashboard Layout	103
6.2.3 Chat Module & Voice Assistant	104
6.2.4 Department Indoor Navigation & QR Scanner Module	105
6.2.5 Profile Management Module	106

LIST OF FIGURES

Figure 1: Agile Development Methodology	32
Figure 2.1: Use Case Model	34
Figure 2.2: SSD - Natural Language Query Submission	45
Figure 2.3: SSD – Voice Based Query Submission	46
Figure 2.4: SSD – User Registration and Login	46
Figure 2.5: SSD – Knowledge Document Upload and RAG Indexing	47
Figure 2.6: SSD – Location and QR Code Management	47
Figure 2.7: Domain Model – ERD	49
Figure 3.1: Class Diagram	52
Figure 3.2: Entity Relationship Diagram (ERD)	54
Figure 3.3: System Architecture Diagram	56
Figure 6.1: Admin Login Screen	98
Figure 6.2: Admin Dashboard Analytics	99
Figure 6.3: User Management Directory	99
Figure 6.4: Add New User Form	100
Figure 6.5: Data Management Overview Screen	100
Figure 6.6: Add Document Portal	101
Figure 6.7: FAQ Management Directory	101
Figure 6.8: Create FAQ Interface	102
Figure 6.9: Student Login Interface	102
Figure 6.10: Student Forgot Password Interface	102
Figure 6.11: OTP Verification Screen	103
Figure 6.12: New Password Screen	103
Figure 6.13: Student Home Dashboard	103
Figure 6.14: AI Chat Interface	104
Figure 6.15: Recent Chats Interface	104
Figure 6.16: AI Chat Processing	104
Figure 6.17: Voice Input Assistant Interface	104
Figure 6.18: Indoor Navigation View	105
Figure 6.19: Integrated QR Code Scanner	105
Figure 6.20: Student Profile Screen	106

LIST OF TABLES

Table 1.1: List of Functional Requirements	10
Table 1.2: FR-01 – User Registration and Login	11
Table 1.3: FR-02 – AI Text Based Query Handling	12
Table 1.4: FR-03 – Voice Input and Speech Output	13
Table 1.5: FR-04 – Multi Turn Conversation Memory	14
Table 1.6: FR-05 – Phonetic and Fuzzy Name Normalization	15
Table 1.7: FR-06 – Internet Search Fallback	16
Table 1.8: FR-07 – Interactive Indoor Navigation Map	17
Table 1.9: FR-08 – QR Code Scanning for Location	18
Table 1.10: FR-09 – Chat History Management	19
Table 1.11: FR-10 – User Account Management	20
Table 1.12: FR-11 – Knowledge Document Upload and Indexing	21
Table 1.13: FR-12 – FAQ Management	22
Table 1.14: FR-13 – Location & QR Code Management	23
Table 1.15: FR-14 – Admin Statistics Dashboard	24
Table 1.16: List of Non-Functional Requirements	26
Table 1.17: Project Schedule – Gantt Chart	29
Table 2.1: Use Case – Natural Language Query Submission	35
Table 2.2: Use Case – Voice Based Query Submission	36
Table 2.3: Use Case – Indoor Navigation with Pathfinding	37
Table 2.4: Use Case – QR Code Scanning for Self-Location	38
Table 2.5: Use Case – User Registration and Login	39
Table 2.6: Use Case – Knowledge Document Upload and RAG Indexing	40
Table 2.7: Use Case – FAQ Management	41
Table 2.8: Use Case – Location and QR Code Management	42
Table 2.9: Use Case – Admin Dashboard Statistics Viewing	44
Table 4.1: Tools and Technologies Used	69
Table 5.1: TC-01	81
Table 5.2: TC-02	82
Table 5.3: TC-03	83
Table 5.4: TC-04	84

Table 5.5: TC-05	85
Table 5.6: TC-06	86
Table 5.7: TC-07	87
Table 5.8: TC-08	88
Table 5.9: TC-09	89
Table 5.10: TC-10	90

ABSTRACT

Universities and academic departments frequently receive repetitive inquiries from students regarding academic policies, departmental procedures, fee structures, course information, faculty details, examination schedules, and campus facilities. New students, in particular, often face difficulties in obtaining authentic information and locating important places within the department. Traditional methods of seeking assistance usually depend on administrative staff, faculty members, or senior students, which can be time-consuming and may not always provide immediate support. To address these challenges, this project presents an AI-Based Virtual Helpdesk for University Department Students, designed to provide intelligent, accurate, and accessible assistance through a mobile application.

The proposed system integrates Artificial Intelligence, Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and modern mobile technologies to create an interactive virtual assistant capable of answering student queries in real time. The chatbot retrieves information from a verified departmental knowledge base, ensuring that responses remain relevant, accurate, and aligned with official departmental resources. In addition to text-based communication, the system also supports voice interaction, enabling users to submit queries through speech and receive responses in a more convenient manner.

A key feature of the system is its smart location assistance module, which helps students find classrooms, laboratories, faculty offices, and other departmental facilities. The system generates QR-based navigation links and provides route guidance to assist users in reaching their desired locations efficiently. An administrative web panel has also been developed to manage FAQs, documents, notices, locations, and other knowledge resources, ensuring that information remains up-to-date and easily accessible.

The system is developed using modern technologies including React Native/Expo for the mobile application, React for the administrative panel, Node.js and Express.js for backend services, MongoDB for data storage, and AI-powered services for intelligent query processing. The implementation demonstrates how AI-driven solutions can enhance student support services, reduce administrative workload, and improve access to departmental information. The proposed virtual helpdesk provides a scalable and user-friendly solution that contributes to the digital transformation of academic support systems within higher education institutions.

CHAPTER 1

SOFTWARE REQUIREMENT SPECIFICATION

This chapter presents the software requirement specification for the AI-Based Virtual Helpdesk for University Department Students. It outlines the purpose, objectives, scope, feasibility analysis, stakeholders, users of the system, functional & non-functional requirements, and schedule of the project providing a comprehensive foundation for understanding the project's intent and boundaries.

1.1 Introduction

The rapid advancement of artificial intelligence has opened new avenues for automating information delivery in academic environments. University departments, particularly those with large student enrollments, face a constant challenge of addressing repetitive and time-sensitive student queries regarding academic procedures, faculty information, course structures, fee rules, and physical navigation within department premises. Manual handling of such queries not only consumes significant administrative time but also leaves students without timely support, especially outside office hours.

This project presents an AI-Based Virtual Helpdesk specifically developed for the Department of Information Technology, Government College University Faisalabad (GCUF). The system is a full-stack, multi-platform solution comprising a React Native mobile application, a Node.js/Express backend server, a Python-based FastAPI AI service, and a React-based admin panel. Students can interact with the system through natural language text or voice queries and receive instant, context-aware responses generated by a Retrieval-Augmented Generation (RAG) pipeline powered by the LLaMA 3.1 language model via the Groq API.

The system's knowledge base is built from verified departmental data including faculty profiles, course structures, semester rules, fee information, and academic policies, all stored as vector embeddings in ChromaDB for semantic retrieval. Beyond conversational assistance, the application provides a smart indoor navigation feature with an SVG-based floor map of the IT department, QR code-based room identification, and graph-based pathfinding to guide students to any room, office, or lab within the department. An administrative panel allows department staff to manage users, FAQs, uploaded knowledge documents, and location data, ensuring the system remains accurate and up to date at all times.

1.2 Project Objectives

1.2.1 General Objectives

- To develop an AI-powered virtual helpdesk that provides students of the IT Department, GCUF, with 24/7 access to authentic and timely departmental information.
- To reduce the burden on department staff by automating responses to frequently asked and repetitive student queries through an intelligent conversational system.
- To integrate smart indoor navigation within the mobile application so students can easily locate classrooms, faculty offices, laboratories, and other rooms within the department.
- To provide a secure and functional administrative panel that enables department staff to manage the system's knowledge base, user accounts, locations, and FAQs.

1.2.2 Specific Objectives

- To implement a Retrieval-Augmented Generation (RAG) pipeline using LangChain, ChromaDB vector database, and the LLaMA 3.1 8B language model (via Groq API) for accurate, context-aware response generation.
- To develop a voice interaction feature using OpenAI Whisper for speech-to-text transcription and Microsoft EdgeTTS for text-to-speech output, enabling fully hands-free query resolution.
- To implement a phonetic and fuzzy text normalization engine that correctly maps colloquial or misspelled faculty name references to canonical names, ensuring query robustness.
- To build an SVG-based interactive indoor map of the IT department floor with BFS/Dijkstra-based pathfinding across a predefined room-node-connection graph covering 25+ locations.
- To implement QR code generation for each department room, enabling students to scan a code outside any room to auto-set their current location on the navigation map.
- To maintain chat session memory by injecting the last four messages as context into each LLM query, enabling coherent multi-turn conversations.
- To provide an internet search fallback via the Tavily API integrated with Google Gemini 2.5 Flash for queries that fall outside the departmental knowledge base.
- To deploy the complete system using Docker with three containerized services: the Node.js backend, the Python AI service, and the React admin panel.

1.3 Scope of the System

The AI-Based Virtual Helpdesk is scoped specifically for the Department of Information Technology at Government College University Faisalabad. The system serves enrolled students of the department as its primary users, while department staff and administrators serve as secondary users through the dedicated admin panel.

On the student-facing side, the mobile application provides two primary modes of interaction: text-based chat and voice-based queries. Students can ask questions about faculty members, their designations, office locations, research publications, course offerings per semester, departmental rules, fee structures, attendance policies, examination procedures, and general departmental operations. The AI system retrieves answers from a verified knowledge base built from department-specific documents and a structured JSON dataset containing faculty and academic information. If a query cannot be answered from the internal knowledge base, the system offers an optional internet search fallback for broader informational queries.

The indoor navigation module is scoped to the IT department floor at GCUF. It covers all classrooms (IT-01 through IT-09), faculty offices, the HOD office, computer labs, the meeting room, the examination centre, and connecting corridors represented as a graph of nodes and connections. Navigation is initiated either manually by selecting a destination or automatically when the AI detects a location-related intent in a student's query. QR codes placed outside each physical room allow students to set their current location by scanning, after which the app computes and displays the optimal path to their destination.

The admin panel is scoped to support full CRUD operations for user management, FAQ management, knowledge document uploads (PDF, DOCX, JSON), location and QR code management, and a statistics dashboard. The system does not cover other departments of GCUF, external university systems, online learning management, or financial transaction processing.

1.4 Feasibility Analysis

There have been discussed Technical Feasibility, Economic Feasibility, and Operational Feasibility in the following sections.

1.4.1 Technical Feasibility

The technical feasibility of this project was assessed by evaluating whether the required technologies, tools, infrastructure, and expertise were available and practically implementable within the project timeline.

The system was built entirely on well-established, open-source, and freely available technologies. The mobile frontend was developed using React Native with Expo, a mature cross-platform framework with extensive community support. The backend server was implemented in Node.js with Express, one of the most widely adopted backend stacks. The AI service was built with Python and FastAPI, both industry-standard choices for machine learning and API development.

Key technical components and their feasibility are as follows:

- **LLM Integration:** The LLaMA 3.1 8B model was accessed via the Groq API, which provides free-tier access with high inference speeds, making it practically feasible without any hardware GPU requirements.
- **RAG Pipeline:** LangChain and ChromaDB were used for orchestration and vector storage respectively. Both are open-source and well-documented, with active community support.
- **Embeddings:** The BAAI/bge-small-en-v1.5 model from HuggingFace was used for generating text embeddings locally, requiring minimal compute resources.
- **Speech Processing:** OpenAI's faster-whisper (base model, CPU/int8) was used for speech-to-text, running efficiently on standard CPU hardware without requiring a GPU.
- **Text-to-Speech:** Microsoft EdgeTTS was used for voice output, operating as a free cloud-based neural TTS service.
- **Indoor Navigation:** The navigation system was implemented using a custom graph-based pathfinding algorithm over a hardcoded room-node-connection map, requiring no third-party mapping services.
- **Containerization:** Docker was used to containerize all three backend services, ensuring consistent and reproducible deployment across environments.

All selected technologies were successfully integrated and tested as part of the completed implementation, confirming full technical feasibility.

1.4.2 Economic Feasibility

The economic feasibility of this project was evaluated by examining the costs associated with development, tools, APIs, infrastructure, and ongoing maintenance.

The project was developed with a strong focus on minimizing costs by leveraging free-tier services, open-source frameworks, and locally hosted components wherever possible. The following summarizes the economic analysis:

- **Development Tools:** All development tools including VS Code, Expo CLI, Node.js, Python, FastAPI, LangChain, ChromaDB, and React were used at no cost under their respective open-source or free licenses.

- **LLM API (Groq):** The Groq API, used to access the LLaMA 3.1 8B model, was utilized under its free tier, which provided sufficient request limits for development, testing, and demonstration purposes.
- **Gemini API (Google):** The Google Gemini 2.5 Flash API, used as a fallback for internet-based queries, was accessed under Google's free tier.
- **Tavily Search API:** The Tavily web search API was used under its free tier for internet fallback queries.
- **Speech Services:** OpenAI Whisper (faster-whisper) was run locally at zero cost, and Microsoft EdgeTTS was used as a free cloud TTS service.
- **Cloudinary:** Image and QR code storage was managed through Cloudinary's free tier, providing adequate storage and bandwidth for the project's scale.
- **MongoDB Atlas:** The database was hosted on MongoDB Atlas under its free M0 cluster tier.
- **Deployment:** Docker-based containerization allows the system to be deployed on any cloud provider or local server. No paid hosting was required for development and demonstration.

Overall, the project was completed with near-zero direct financial expenditure, making it highly economically feasible for an academic final year project. For future production deployment serving a full student body, minimal costs would be incurred primarily for cloud hosting and API usage scaling, which remain economically justified given the operational value delivered.

1.4.3 Operational Feasibility

Operational feasibility assesses whether the proposed system can be effectively used by its intended users within the existing organizational environment of the IT Department, GCUF.

The system was designed with student usability as the primary concern. The mobile application features a simple, intuitive interface with clearly separated sections for chat, voice interaction, indoor navigation, and QR scanning. New students, who are the primary target audience, can begin using the helpdesk without any training, as the interface follows familiar mobile app conventions. The onboarding screens introduced at first launch guide new users through the application's features.

From an operational standpoint, the following factors confirm feasibility:

- **24/7 Availability:** The AI-powered system operates continuously without requiring human staff availability, addressing the core problem of after-hours query support.
- **Low Maintenance Overhead:** The admin panel provides a straightforward interface for department staff to update FAQs, upload new documents, and manage locations without requiring technical expertise.

- **Scalability of Knowledge Base:** Department staff can upload new PDFs, DOCX files, or JSON datasets through the admin panel at any time, and the system automatically indexes them into the vector database for immediate use.
- **Voice Accessibility:** The voice interaction feature makes the system accessible to students who prefer spoken queries, broadening its usability.
- **Language and Query Robustness:** The phonetic and fuzzy name normalization engine ensures that queries with misspelled or colloquially written faculty names are still correctly resolved, reducing user frustration.
- **Physical Integration:** QR codes placed outside department rooms physically integrate the digital navigation system with the real-world department environment, making location features immediately usable upon deployment.
- **Admin Control:** Department administrators retain full control over all system data through the admin panel, ensuring the system remains aligned with actual departmental information at all times.

Given these factors, the system is operationally feasible and practical for immediate deployment within the IT Department of GCUF.

1.5 Stakeholders

The stakeholders of the AI-Based Virtual Helpdesk are all individuals and groups who have a direct or indirect interest in the development, deployment, operation, or outcomes of the system.

The primary stakeholder is the **Department of Information Technology, GCUF**, which commissioned this solution to modernize student support services and reduce the administrative burden on faculty and staff. The department has a direct interest in ensuring the system accurately represents departmental information, policies, and personnel, and that it operates reliably for the student body.

The **students of the IT Department** are the most direct beneficiaries of the system. They interact with it daily to resolve academic queries, locate facilities, and access departmental information. Their satisfaction and ease of use are central to the system's success. This group particularly includes newly enrolled students who are unfamiliar with departmental procedures, locations, and faculty.

Faculty members of the IT Department are indirect stakeholders whose information — including office locations, designations, course assignments, and research publications — forms a significant portion of the system's knowledge base. Accurate representation of faculty data directly affects how students are served.

The **department administration and staff**, including the Head of Department (HOD) and administrative officers, are stakeholders who interact with the system through the admin panel. They are responsible for keeping the knowledge base current, managing

user accounts, and overseeing location and FAQ data. Their operational ease directly impacts the long-term sustainability of the system.

Finally, the **project development team** — M. Bilal Aqeel and Abdul Waleed — and their **FYP supervisor** are academic stakeholders who have a vested interest in the system's technical correctness, completeness, and alignment with the stated objectives and academic requirements.

1.6 Users of the System

The AI-Based Virtual Helpdesk serves two distinct categories of users. Each category interacts with a different interface, holds different access privileges, and fulfills a different role within the system's operational structure. The system was designed to ensure that both user types can interact with their respective interfaces intuitively, without requiring any specialized technical knowledge.

1.6.1 Students

Students are the primary and most frequent end users of the system. They access the helpdesk through the React Native mobile application, compatible with both Android and iOS platforms. A student account is identified by the university registration number and is created either through self-registration or by an administrator. The following describes the key interactions and capabilities available to a student user:

- **AI Chat:** Students can type natural language queries about faculty, courses, semester rules, fee structures, attendance policies, examination procedures, and any other department-related topic, and receive instant AI-generated responses.
- **Voice Interaction:** Students can submit queries through voice input. The system transcribes the spoken query using Whisper, processes it through the RAG pipeline, and delivers the response both as text and as synthesized speech via EdgeTTS.
- **Indoor Navigation:** Students can select any room, office, or lab within the IT department on the interactive SVG floor map and receive a computed navigation path from their current location to the destination.
- **QR Code Scanning:** Students can scan QR codes placed outside department rooms to automatically set their current location on the map, after which they can select any destination and get directions.
- **Internet Fallback:** If the AI's response is not found in the departmental knowledge base, students can optionally trigger an internet search for broader informational support.

- **Chat History:** Students can view their previous conversation sessions stored against their account.
- **Profile Management:** Students can view and update their profile, including uploading a profile picture.

1.6.2 Administrators

Administrators are the secondary users of the system. This category includes department staff, administrative officers, and the Head of Department (HOD) who are granted elevated access. Administrators interact with the system primarily through the React-based admin web panel, as well as through the mobile application where applicable. Administrators are responsible for the accuracy, completeness, and currency of all data that drives the AI system. The following outlines the administrative capabilities:

- **User Management:** Administrators can add new student or admin accounts, update existing user information, reset passwords, and delete accounts from the system.
- **Knowledge Document Management:** Administrators can upload departmental documents in PDF, DOCX, or JSON format through the admin panel. Upon upload, the system automatically processes, chunks, embeds, and indexes the documents into the ChromaDB vector database for immediate use by the RAG pipeline.
- **FAQ Management:** Administrators can create, update, and delete FAQs organized across multiple categories including general, technical, fee, admissions, attendance, and results, providing structured supplementary content for the AI.
- **Location and QR Code Management:** Administrators can add department room locations with relevant metadata. Upon saving, the system automatically generates a unique QR code for each room, uploads it to Cloudinary, and makes it available for printing and physical placement outside the room.
- **Dashboard Statistics:** Administrators have access to a statistics dashboard displaying an overview of total registered users, total students, total uploaded documents, and total FAQs, giving a quick operational snapshot of the system.

1.7 Functional Requirements

The functional requirements of the AI-Based Virtual Helpdesk define all the specific behaviours, operations, and capabilities that the system must perform to fulfil its intended purpose. These requirements capture what the system does — from how a student interacts with the AI chatbot to how an administrator manages the knowledge base and user accounts. Each functional requirement is derived from the project objectives, stakeholder

needs, and the defined scope of the system. The requirements cover all four major components of the system: the student-facing mobile application, the AI and RAG processing pipeline, the indoor navigation and QR module, and the administrator web panel.

1.7.1 List of Functional Requirements

The system encompasses a comprehensive set of functional requirements distributed across its core modules. These requirements were identified through analysis of the problem domain, stakeholder expectations, and the technical capabilities of the implemented system. The primary functional areas include user authentication and account management, AI-powered conversational query handling, voice interaction, indoor navigation with QR code support, internet search fallback, chat history management, knowledge document management, FAQ management, and the administrative dashboard.

The following key functional requirements govern the system's behavior:

- The system shall allow students to log in using their university registration number and password.
- The system shall allow students to submit natural language text queries and receive AI-generated, context-aware responses from the departmental knowledge base via the RAG pipeline.
- The system shall support voice input from students, transcribing speech using OpenAI Whisper and delivering spoken responses via Microsoft EdgeTTS.
- The system shall provide an interactive SVG-based indoor floor map of the IT Department, GCUF, with BFS/Dijkstra pathfinding to compute and display optimal navigation routes.
- The system shall generate unique QR codes for each department room, enabling students to scan and auto-set their current location on the map.
- The system shall maintain multi-turn conversation context by injecting the last four messages as memory into each LLM query.
- The system shall provide an internet search fallback via Tavily API and Google Gemini 2.5 Flash for queries outside the departmental knowledge base.
- The system shall allow students to view their previous chat session history.
- The system shall allow administrators to manage user accounts (create, update, delete).
- The system shall allow administrators to upload knowledge documents (PDF, DOCX, and JSON), which the system automatically chunks, embeds, and indexes into ChromaDB.
- The system shall allow administrators to manage FAQs across multiple categories.
- The system shall allow administrators to add, update, and delete department room locations and auto-generate their QR codes.
- The system shall provide administrators with a statistics dashboard showing total users, students, documents, and FAQs.

- The system shall apply phonetic and fuzzy text normalization to correctly resolve misspelled or colloquial faculty name queries.

Table 1.1: List of Functional Requirements

Req #	Requirement Name	Module	User Role	Priority
FR-01	User Registration & Login	Authentication	Student / Admin	High
FR-02	AI Text-Based Query Handling	AI Chatbot	Student	High
FR-03	Voice Input & Speech Output	Voice Interaction	Student	High
FR-04	Multi-Turn Conversation Memory	AI Chatbot	Student	High
FR-05	Phonetic & Fuzzy Name Normalization	AI Processing	Student	Medium
FR-06	Internet Search Fallback	AI Chatbot	Student	Medium
FR-07	Interactive Indoor Navigation Map	Navigation	Student	High
FR-08	QR Code Scanning for Location	Navigation	Student	High
FR-09	Chat History Management	Chat Module	Student	Medium
FR-10	User Account Management (Admin)	Admin Panel	Administrator	High
FR-11	Knowledge Document Upload & Indexing	Admin Panel	Administrator	High
FR-12	FAQ Management	Admin Panel	Administrator	Medium
FR-13	Location & QR Code Management	Admin Panel	Administrator	High
FR-14	Admin Statistics Dashboard	Admin Panel	Administrator	Medium

1.7.2 Description of Each Requirement in Requirement Shell

The following subsection presents a detailed description of each functional requirement using the standard Requirement Shell format. Each shell captures the requirement's purpose, rationale, originator, fit criterion, priority, and conflict status, providing a complete and traceable specification for each feature of the system.

Requirement Name: User Registration & Login**Table 1.2: FR-01 — User Registration & Login**

Requirement #:	FR-01
Requirement Type	Functional
Description:	The system shall allow students to log in using their university registration number, and password. Administrators can create accounts from the admin panel. Both students and admins shall be able to log in securely using JWT-based authentication. Sessions shall be maintained across app restarts.
Rationale:	Secure authentication ensures that only authorized students and administrators can access the system, protecting student data and administrative controls.
Originator:	Students, Department Administration
Fit Criterion:	A student can successfully log in to access the mobile app dashboard. An administrator can log in to the admin panel and access all management features. JWT tokens are issued and validated correctly.
Customer Satisfaction:	5
Customer Dissatisfaction	5
Priority:	High
Supporting Materials:	MongoDB Users Collection, JWT Authentication Middleware
History:	Identified during requirements gathering; implemented and tested in Sprint 1

Requirement Name: AI Text-Based Query Handling**Table 1.3: FR-02 — AI Text-Based Query Handling**

Requirement #:	FR-02
Requirement Type	Functional
Description:	The system shall allow students to submit natural language text queries through the mobile chat interface. Queries shall be processed by the RAG pipeline, which retrieves semantically relevant chunks from the ChromaDB vector database and passes them along with the query to the LLaMA 3.1 8B model via the Groq API to generate accurate, context-aware responses.
Rationale:	The core purpose of the helpdesk is to instantly answer student queries about faculty, courses, rules, fees, and departmental procedures without requiring human staff involvement.
Originator:	Students
Fit Criterion:	A student submits a query and receives a relevant, accurate response within an acceptable response time. The system correctly retrieves context from the knowledge base and generates a coherent answer using the LLM.
Customer Satisfaction:	5
Priority:	High
Supporting Materials:	ChromaDB Vector DB, LangChain RAG Pipeline, Groq API, LLaMA 3.1 8B
History:	Core requirement; implemented and tested across Sprints 2–3

Requirement Name: Voice Input & Speech Output**Table 1.4: FR-03 — Voice Input & Speech Output**

Requirement #:	FR-03
Requirement Type	Functional
Description:	The system shall allow students to submit queries through voice input. The mobile app shall record the student's speech, send the audio to the AI service, which transcribes it using OpenAI Whisper (faster-whisper, base model). The transcribed query shall be processed through the RAG pipeline and the response shall be delivered both as text on screen and as synthesized speech via Microsoft EdgeTTS.
Rationale:	Voice interaction makes the helpdesk accessible to students in situations where typing is inconvenient, and provides a more natural, hands-free mode of interaction.
Originator:	Students
Fit Criterion:	A student records a voice query, the system correctly transcribes it, processes it through the AI pipeline, and delivers both a text and audio response. Transcription accuracy is acceptable for standard spoken queries in English.
Customer Satisfaction:	4
Priority:	High
Supporting Materials:	OpenAI Whisper (faster-whisper), Microsoft EdgeTTS
History:	Identified in proposal; implemented in Sprint 3

Requirement Name: Multi-Turn Conversation Memory**Table 1.5: FR-04 — Multi-Turn Conversation Memory**

Requirement #:	FR-04
Requirement Type	Functional
Description:	The system shall maintain conversational context across multiple turns within a chat session. The last four messages (two from the student and two from the assistant) shall be injected as context into each new LLM query, enabling the model to understand follow-up questions and maintain coherent dialogue.
Rationale:	Without conversation memory, the chatbot would treat every message as an isolated query, making it unable to handle follow-up questions or contextual references, which significantly degrades the user experience.
Originator:	Students
Fit Criterion:	A student can ask a follow-up question that references a previous answer, and the system provides a contextually accurate response demonstrating awareness of prior messages.
Customer Satisfaction:	4
Priority:	High
Supporting Materials:	LangChain Memory Buffer, RAG Pipeline
History:	Identified during design phase; implemented in Sprint 3

Requirement Name: Phonetic & Fuzzy Name Normalization**Table 1.6: FR-05 — Phonetic & Fuzzy Name Normalization**

Requirement #:	FR-05
Requirement Type	Functional
Description:	The system shall apply a phonetic and fuzzy text normalization engine to preprocess student queries. The engine shall detect misspelled, abbreviated, or colloquially written faculty name references and map them to canonical faculty names before passing the query to the RAG pipeline.
Rationale:	Students often write faculty names incorrectly due to spelling variations, abbreviations, or phonetic substitutions. Without normalization, the system would fail to retrieve accurate information about the queried faculty member.
Originator:	Students, Project Team
Fit Criterion:	A query containing a misspelled or colloquial faculty name is correctly normalized to the canonical name and the system retrieves accurate faculty-related information in its response.
Customer Satisfaction:	4
Priority:	Medium
Supporting Materials:	Custom NLP Preprocessing Module, Faculty Name Dataset
History:	Identified during implementation; implemented in Sprint 3

Requirement Name: Internet Search Fallback**Table 1.7: FR-06 — Internet Search Fallback**

Requirement #:	FR-06
Requirement Type	Functional
Description:	When the RAG pipeline cannot find a sufficiently relevant answer in the departmental knowledge base, the system shall offer students an option to trigger an internet search. The search shall be performed via the Tavily API, and the result shall be processed by Google Gemini 2.5 Flash to generate a summarized, readable response.
Rationale:	Not all student queries are strictly departmental in nature. Providing a fallback ensures students receive useful information even for queries beyond the local knowledge base, without leaving the application.
Originator:	Students
Fit Criterion:	When no relevant departmental answer is found, the student is presented with a fallback option. Upon triggering it, the system performs a web search and returns a coherent summarized answer within an acceptable time.
Customer Satisfaction:	3
Priority:	Medium
Supporting Materials:	Tavily API, Google Gemini 2.5 Flash API
History:	Identified during design; implemented in Sprint 4

Requirement Name: Interactive Indoor Navigation Map**Table 1.8: FR-07 — Interactive Indoor Navigation Map**

Requirement #:	FR-07
Requirement Type	Functional
Description:	The system shall provide an SVG-based interactive floor map of the IT Department, GCUF, covering 25+ locations including classrooms (IT-01 to IT-09), faculty offices, HOD office, computer labs, meeting room, examination centre, and corridors. Students shall be able to select their current location and a destination, and the system shall compute and display the optimal path using BFS/Dijkstra pathfinding on the room-node-connection graph. Navigation can also be triggered automatically when the AI detects a location-related intent in a student's chat query.
Rationale:	New students frequently struggle to locate rooms and offices within the department. A built-in navigation system directly addresses this problem, reducing confusion and physical dependence on others.
Originator:	Students
Fit Criterion:	A student selects a source and destination on the map and the system highlights the correct optimal path. The navigation is also triggered correctly when a location intent is detected in a chat query.
Customer Satisfaction:	5
Priority:	High
Supporting Materials:	SVG Floor Map, BFS/Dijkstra Graph Algorithm, Room-Node-Connection Dataset

History:	Identified in proposal as a core module; implemented in Sprint 4
-----------------	--

Requirement Name: QR Code Scanning for Location

Table 1.9: FR-08 — QR Code Scanning for Location

Requirement #:	FR-08
Functional Type	Functional
Description:	The system shall allow students to scan QR codes placed outside department rooms using the mobile app's camera. Upon scanning, the app shall automatically set the student's current location on the indoor navigation map to the scanned room, after which the student can select a destination and receive navigation directions.
Rationale:	Manually identifying one's current location on a map can be confusing, especially for new students. QR code scanning provides an instant, error-free mechanism for self-location on the map.
Originator:	Students
Fit Criterion:	A student scans a QR code outside a room and the app correctly sets that room as the current location on the navigation map. The student can then select a destination and receive correct directions.
Customer Satisfaction:	5
Priority:	High
Supporting Materials:	QR Scanner Package (React Native), Cloudinary (QR Storage), Location Collection (MongoDB)

History:	Identified in proposal; implemented in Sprint 4
-----------------	---

Requirement Name: Chat History Management

Table 1.10: FR-09 — Chat History Management

Requirement #:	FR-09
Requirement Type	Functional
Description:	The system shall persist all chat conversations for each student account in the database. Students shall be able to view a list of their previous chat sessions and browse the full message history of any past session.
Rationale:	Persisting chat history allows students to revisit previous answers without needing to ask the same question again, improving convenience and reducing repeated load on the AI pipeline.
Originator:	Students
Fit Criterion:	A student can navigate to their chat history screen and view all previous sessions with their complete message threads, correctly ordered by date.
Customer Satisfaction:	3
Priority:	Medium
Supporting Materials:	MongoDB Chat Sessions Collection, Node.js Backend API
History:	Identified during requirements gathering; implemented in Sprint 2

Requirement Name: User Account Management (Admin)

Table 1.11: FR-10 — User Account Management

Requirement #:	FR-10
Requirement Type	Functional
Description:	The system shall allow administrators to perform full CRUD operations on user accounts through the admin web panel. Admins can create new student or admin accounts, update user information, reset passwords, and delete accounts from the system.
Rationale:	Centralized user management ensures that only authorized individuals have access to the system and that account information remains accurate and up to date.
Originator:	Department Administration
Fit Criterion:	An administrator can successfully create, update, and delete a user account. Changes are reflected immediately in the database and the affected user's access is updated accordingly.
Customer Satisfaction:	4
Priority:	High
Supporting Materials:	MongoDB Users Collection, Node.js REST API, Admin React Panel
History:	Identified during requirements gathering; implemented in Sprint 1

Requirement Name: Knowledge Document Upload & Indexing**Table 1.12: FR-11 — Knowledge Document Upload & Indexing**

Requirement #:	FR-11
Requirement Type	Functional
Description:	The system shall allow administrators to upload departmental documents in PDF, DOCX, or JSON format through the admin panel. Upon upload, the Python AI service shall automatically process the document — chunking the content, generating vector embeddings using the BAAI/bge-small-en-v1.5 model, and indexing the chunks into the ChromaDB vector database — making the content immediately available to the RAG pipeline.
Rationale:	The accuracy and breadth of the AI's responses depend entirely on the quality and currency of the knowledge base. A seamless upload and auto-indexing mechanism ensures administrators can keep the knowledge base up to date without technical involvement.
Originator:	Department Administration
Fit Criterion:	An administrator uploads a document and within a reasonable time the document's content is retrievable through student chat queries. The document appears in the admin document list with correct metadata.
Customer Satisfaction:	5
Priority:	High

Supporting Materials:	FastAPI AI Service, LangChain Document Loaders, BAAI/bge-small-en-v1.5, ChromaDB
History:	Core requirement; implemented in Sprint 2–3

Requirement Name: FAQ Management

Table 1.13: FR-12 — FAQ Management

Requirement #:	FR-12
Requirement Type	Functional
Description:	The system shall allow administrators to create, update, and delete FAQs organized across multiple categories including General, Technical, Fee, Admissions, Attendance, and Results. FAQs serve as structured supplementary content that enriches AI responses for common, well-defined queries.
Rationale:	FAQs provide a fast, structured layer of knowledge for the most common and predictable student queries, reducing RAG retrieval overhead and ensuring consistent answers for well-known topics.
Originator:	Department Administration
Fit Criterion:	An administrator can add a new FAQ under a selected category, update its content, and delete it. The FAQ is reflected in the knowledge base and influences relevant AI responses.
Customer Satisfaction:	3
Priority:	Medium
Supporting Materials:	MongoDB FAQs Collection, Node.js REST API, Admin React Panel

History:	Identified during requirements gathering; implemented in Sprint 2
-----------------	---

Requirement Name: Location & QR Code Management

Table 1.14: FR-13 — Location & QR Code Management

Requirement #:	FR-13
Requirement Type	Functional
Description:	The system shall allow administrators to add, update, and delete department room locations, each with relevant metadata such as room name, floor, type (classroom, lab, office), and description. Upon saving a new location, the system shall automatically generate a unique QR code for that room, upload it to Cloudinary, and store the Cloudinary URL in the database. The QR code can then be downloaded and physically placed outside the room.
Rationale:	Location data powers both the navigation map and QR scanning features. Centralized admin management of locations and auto-generation of QR codes ensures consistency between physical room labels and the digital navigation system.
Originator:	Department Administration
Fit Criterion:	An administrator adds a new room location and the system automatically generates and uploads a QR code linked to that room. The QR code is scannable by the mobile app and correctly maps to the room on the navigation map.

Customer Satisfaction:	4
Priority:	High
Supporting Materials:	Cloudinary API, MongoDB Locations Collection, QR Code Generator Library
History:	Identified in proposal; implemented in Sprint 4

Requirement Name: Admin Statistics Dashboard

Table 1.15: FR-14 — Admin Statistics Dashboard

Requirement #:	FR-14
Requirement Type	Functional
Description:	The system shall provide administrators with a statistics dashboard upon login that displays an overview of key system metrics including total registered users, total students, total uploaded knowledge documents, and total FAQs currently in the system.
Rationale:	A dashboard gives department administrators an at-a-glance operational view of the system's data status, enabling informed decisions about knowledge base maintenance and user management.
Originator:	Department Administration
Fit Criterion:	The admin dashboard correctly displays accurate counts for all four metrics, consistent with the actual data stored in the database at the time of viewing.
Customer Satisfaction:	3
Priority:	Medium

Supporting Materials:	MongoDB Aggregation Queries, Node.js REST API, Admin React Panel
History:	Identified during design; implemented in Sprint 1

1.8 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that the AI-Based Virtual Helpdesk must satisfy in addition to its functional behavior. While functional requirements specify what the system does, non-functional requirements specify how well it does it — addressing dimensions such as performance, security, usability, reliability, maintainability, scalability, and portability. These requirements are critical to ensuring that the system not only delivers the correct outputs but does so in a manner that is responsive, secure, and sustainable for long-term use within the IT Department of GCUF.

The following non-functional requirements have been identified and must be met by the implemented system:

- The system shall deliver AI chatbot responses within an acceptable response time under normal operating conditions, ensuring students do not experience frustrating delays during query resolution.
- The system shall implement JWT-based authentication and enforce role-based access control to ensure that student and admin functionalities remain strictly separated and secure.
- The mobile application shall provide an intuitive, user-friendly interface that requires no prior training for new students to begin using the helpdesk effectively.
- The system shall be available 24 hours a day, 7 days a week, enabling students to access information at any time, including outside regular department office hours.
- The system shall be deployable using Docker containers, ensuring consistent, reproducible deployment across different environments and cloud platforms.
- The system's knowledge base shall be extensible, allowing administrators to upload new documents without any code changes or system restarts, ensuring the AI remains current.
- The mobile application shall be compatible with both Android and iOS platforms through the React Native/Expo framework.
- All sensitive data, including user passwords and authentication tokens, shall be stored and transmitted securely using industry-standard encryption practices.
- The system's RAG pipeline and vector database shall be capable of scaling to accommodate a growing knowledge base without significant degradation in retrieval accuracy or speed.

- The codebase shall be structured in a maintainable and modular fashion across its four service layers — mobile app, Node.js backend, Python AI service, and React admin panel — to facilitate future updates and extensions.

Table 1.16: List of Non-Functional Requirements

NFR #	Requirement Name	Category	Description	Priority
NFR-01	Response Performance	Performance	The AI chatbot shall return a response within an acceptable time under standard load. The system shall handle concurrent user sessions without significant degradation.	High
NFR-02	Authentication Security	Security	All user sessions shall be secured using JWT tokens. Passwords shall be stored as salted hashes. Role-based access control shall prevent unauthorized access to admin features.	High
NFR-03	Data Transmission Security	Security	All API communication between the mobile app, backend, and AI service shall occur over HTTPS. Sensitive data shall never be transmitted in plain text.	High
NFR-04	Usability	Usability	The mobile application shall provide an intuitive interface following standard mobile UX conventions. New students shall be able to use the chatbot and navigation features without prior training or documentation.	High
NFR-05	System Availability	Reliability	The system shall maintain 24/7 availability. Containerized deployment via Docker shall ensure that service failures in one container do not bring down the entire system.	High
NFR-06	Cross-Platform Compatibility	Portability	The mobile application shall run on both Android and iOS devices using the React Native/Expo framework without platform-specific code changes.	High
NFR-07	Knowledge Base Extensibility	Maintainability	Administrators shall be able to upload new documents at any time through the admin panel. The system shall automatically	High

			process and index new content without requiring code changes or system restarts.	
NFR-08	Scalability	Scalability	The ChromaDB vector database and RAG pipeline shall support a growing knowledge base. The Node.js backend shall be capable of handling increased user load through horizontal scaling.	Medium
NFR-09	Deploy ability	Portability	The system shall be fully containerized using Docker with three independently deployable services: Node.js backend, Python AI service, and React admin panel. Deployment shall be reproducible across any compatible cloud or local environment.	Medium
NFR-10	Maintainability	Maintainability	The system shall follow a modular, layered architecture with clearly separated concerns across services. Code shall follow consistent conventions to support future modifications and feature additions by other developers.	Medium
NFR-11	Offline Graceful Degradation	Reliability	The mobile application shall display appropriate error messages when network connectivity is unavailable, rather than crashing or presenting unhandled errors to the student.	Low
NFR-12	Voice Processing Accuracy	Performance	The Whisper-based speech-to-text module shall achieve acceptable transcription accuracy for standard English queries spoken in a normal environment, sufficient for correct intent recognition by the RAG pipeline.	Medium

1.9 Schedule of the Project

The AI-Based Virtual Helpdesk project was executed following an agile development methodology, with work organized into iterative sprints spanning the full project duration from March 2026 to June 2026. The Agile approach was selected to allow for continuous refinement of requirements, regular testing cycles, and the flexibility to incorporate feedback from stakeholders throughout the development process. The project lifecycle was divided into six major phases: project planning and requirements engineering, system design and architecture, core backend and AI service development, mobile application development, admin panel development, and system integration, testing, and final documentation.

The project schedule was structured to ensure that foundational components — particularly the RAG pipeline, database schema, and authentication framework — were established in early sprints before the more complex navigation, voice, and QR features were layered on top. Testing and documentation activities were carried out concurrently with development in later sprints rather than treated as a terminal phase, ensuring quality was maintained throughout the project lifecycle.

Key milestones of the project schedule include:

- **Phase 1 (21 March – 25 March):** Project planning, literature review, requirements elicitation, and proposal finalization.
- **Phase 2 (26 March – 31 March):** System architecture design, database schema design, UI/UX wireframing using Figma, and technology stack finalization.
- **Phase 3 (01 April – 15 April):** Backend API development (Node.js/Express), MongoDB database setup, JWT authentication implementation, and FastAPI AI service scaffold.
- **Phase 4 (16 April – 30 April):** RAG pipeline implementation (LangChain, ChromaDB, LLaMA 3.1 via Groq), knowledge base ingestion, and embedding model integration.
- **Phase 5 (01 May – 10 May):** React Native mobile app development — chat interface, voice interaction (Whisper + EdgeTTS), indoor navigation SVG map, QR scanning, and chat history.
- **Phase 6 (11 May – 25 May):** React admin panel development — user management, document upload, FAQ management, location/QR management, and dashboard.
- **Phase 7 (26 May – 05 June):** System integration, end-to-end testing, bug fixing, Docker containerization, and performance optimization.
- **Phase 8 (06 June – 15 June):** Final documentation, FYP report writing, demonstration preparation, and project submission.

Table 1.17: Project Schedule — Gantt Chart

Phase	Activity	21 March – 25 March 2026	26 March – 31 March 2026	01 April – 15 April 2026	16 April – 30 April 2026	01 May – 10 May 2026	11 May – 25 May 2026	26 May – 05 June 2026	06 June – 15 June 2026
Phase 1	Project Planning & Requirements	✓							
Phase 1	Literature Review & Proposal	✓							
Phase 2	System & Database Design		✓						
Phase 2	UI/UX Wireframing (Figma)		✓						
Phase 3	Backend API Development (Node.js)			✓					
Phase 3	JWT Auth & MongoDB Setup			✓					
Phase 3	FastAPI AI Service Scaffold			✓					

Phase 4	RAG Pipeline (LangChain + ChromaDB)				✓				
Phase 4	LLM Integration (LLaMA 3.1 / Groq)				✓				
Phase 4	Knowledge Base Ingestion & Embeddings				✓				
Phase 5	Mobile App — Chat Interface					✓			
Phase 5	Voice Interaction (Whisper + EdgeTTS)					✓			
Phase 5	Indoor Navigation & QR Scanning					✓			
Phase 5	Internet Fallback (Tavily + Gemini)					✓			
Phase 6	Admin Panel — User & Doc Management						✓		

Phase 6	Admin Panel — FAQ & Location/QR Mgmt								
Phase 6	Admin Dashboard & Statistics								
Phase 7	System Integration & E2E Testing								
Phase 7	Bug Fixing & Performance Optimization								
Phase 7	Docker Containerization & Deployment								
Phase 8	FYP Documentation & Report Writing								
Phase 8	Demonstration & Project Submission								

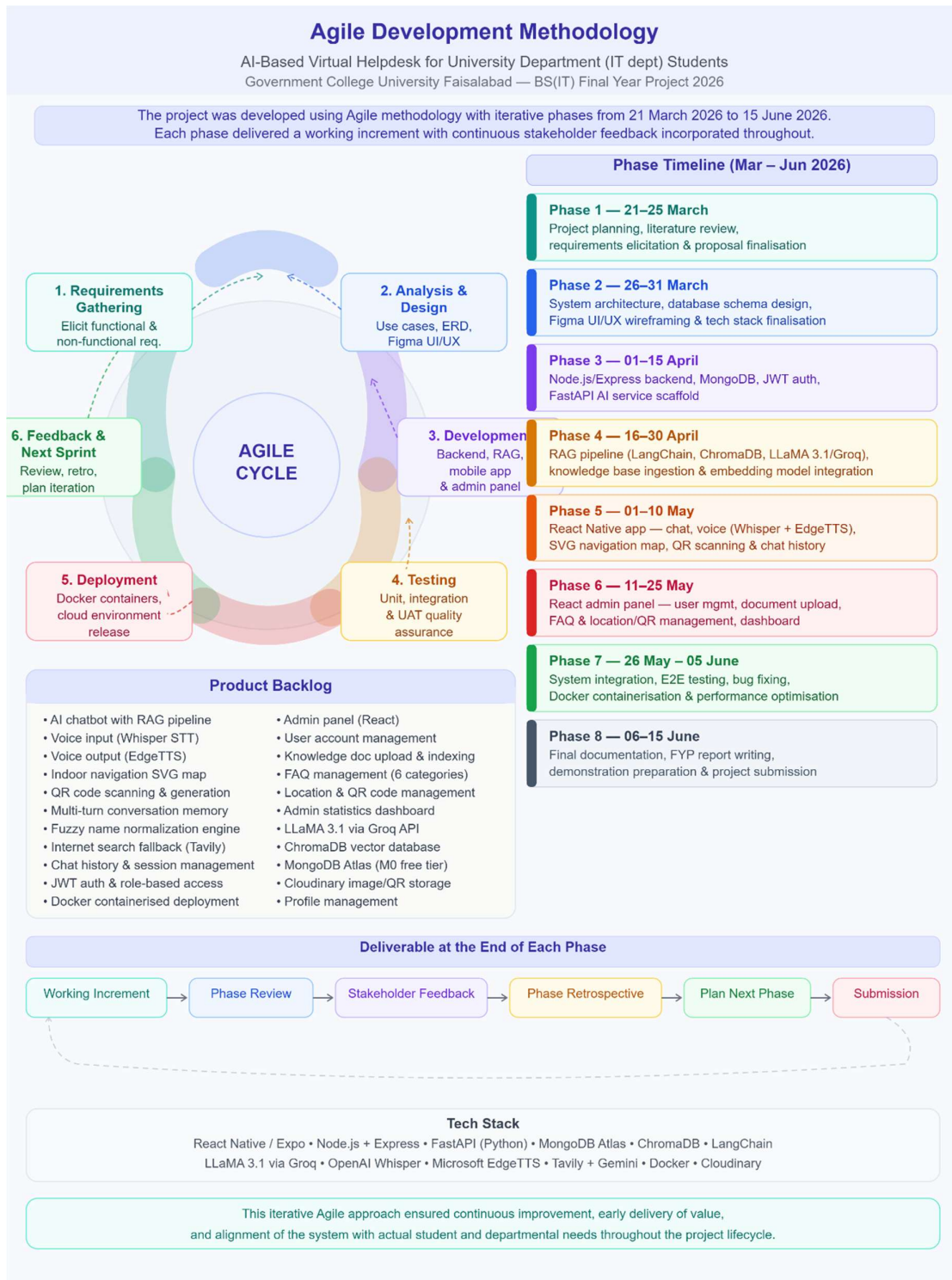


Figure 1: Agile Development Methodology

CHAPTER 2

PROJECT ANALYSIS

This chapter presents the analysis of the AI-Based Virtual Helpdesk system through the lens of object-oriented requirements analysis. It identifies the actors that interact with the system, models their goals as use cases, describes each use case in fully dressed format, illustrates the message-level interactions between actors and the system through System Sequence Diagrams, and concludes with a Domain Model representing the core conceptual entities of the system and their relationships.

2.1 Use Case Model

The use case model identifies the two primary actors of the AI-Based Virtual Helpdesk — the Student and the Administrator — and the set of goal-oriented interactions each performs with the system. Students represent the primary end users who interact with the system through the React Native mobile application to obtain departmental information, navigate the department physically, and manage their own account. Administrators represent department staff who interact through the React-based admin panel to maintain the knowledge base, manage users, FAQs, and location data, and monitor system usage through the dashboard.

The use case diagram below groups these interactions within the system boundary and shows two external systems that the helpdesk depends on: the Groq-hosted LLaMA 3.1 language model, which powers the Retrieval-Augmented Generation pipeline that answers student queries, and the Gemini + Tavily combination, which is invoked as an internet search fallback when the departmental knowledge base does not contain a sufficient answer.

Key use cases identified for the system include:

- **Register / Login** — students self-register or log in using their registration number; admins log in with email and can reset passwords via OTP.
- **Ask AI chatbot** — students submit natural-language text queries answered by the RAG pipeline.
- **Voice interaction** — students submit spoken queries transcribed via Whisper and receive spoken replies via EdgeTTS.
- **View indoor navigation** — students view the SVG floor map and get a computed shortest path between two rooms.
- **Scan QR for location** — students scan a room's QR code to auto-set their current location on the navigation map.
- **View chat history** — students browse their past conversation sessions.

- **Use internet fallback** — students trigger a web search when the knowledge base lacks an answer.
- **Manage profile** — students update their profile picture and personal details.
- **Manage user accounts, FAQs, locations/QR codes, and knowledge documents** — administrators perform CRUD operations to keep the system's data current.
- **View dashboard statistics** — administrators view an operational overview of users, documents, and FAQs.

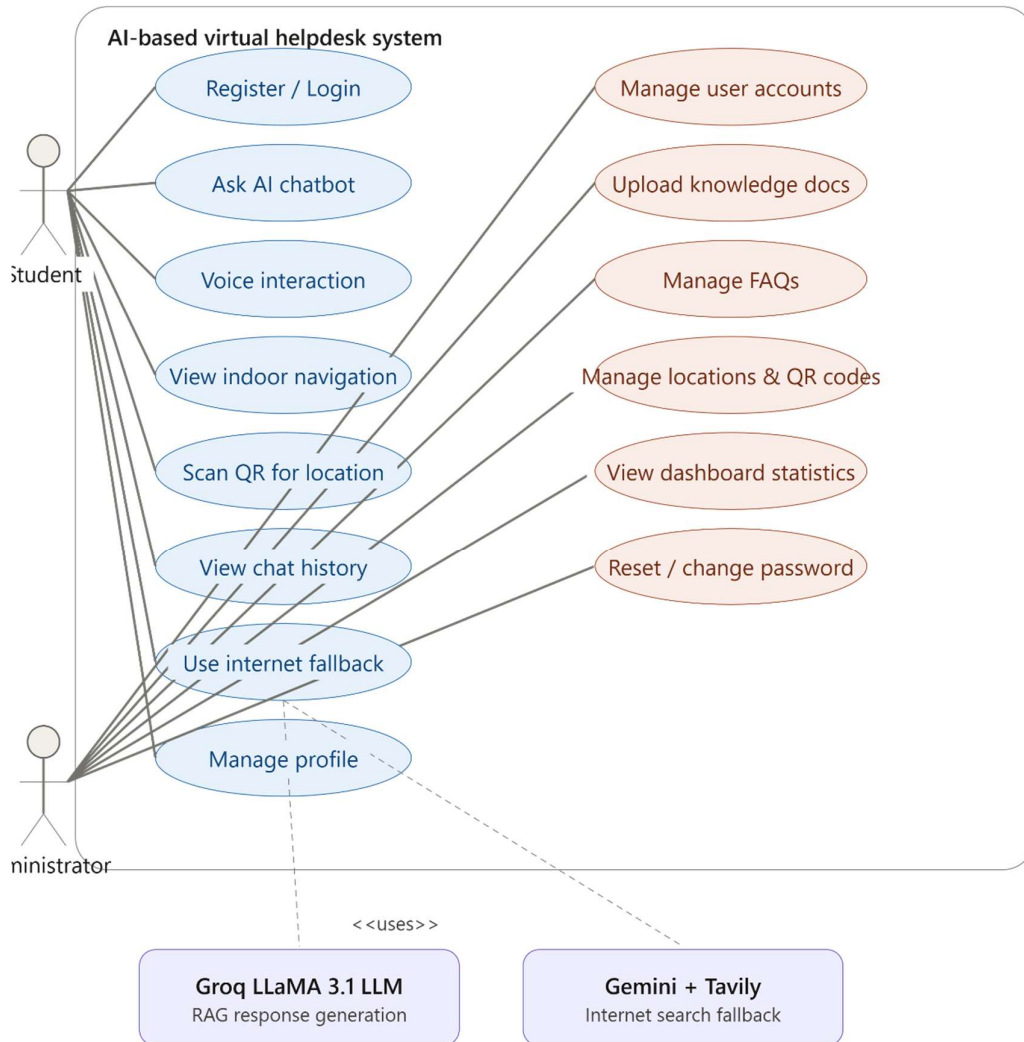


Figure 2.1: Use Case Model

2.2 Description of Each Use Case in Fully Dressed Format

Below are nine use cases describing the system's core behavior in fully dressed format.

2.2.1 Use Case UC-01: Natural Language Query Submission (AI Chatbot)

Table 2.1: Use Case — Natural Language Query Submission

Field	Description
UC #	UC-01 — Ref: FR-02, FR-04, FR-05, FR-06
UC Name	Natural Language Query Submission
Level	Detailed
Description	A logged-in student submits a natural-language text query about the IT Department. The system detects intent, retrieves relevant chunks from ChromaDB, generates a context-aware response via the Groq LLaMA 3.1 model, and saves the exchange to chat history.
Actor(s)	Student
Stakeholders	Students, Department Administration, FYP Supervisor
Preconditions	Student is authenticated with a valid JWT token. The knowledge base contains at least one indexed document.
Success Guarantee	The student receives an accurate, context-grounded response, and the exchange is persisted in the chat session.
Main Success Scenario	1. Student types a query in the chat interface and submits it. 2. Mobile app sends the query, chat ID (if any), and <code>use_internet</code> flag to the Node.js backend. 3. Backend retrieves the last four messages from the chat for memory context and forwards the request to the Python AI service. 4. AI service normalizes the query and runs phonetic/fuzzy teacher-name normalization. 5. Intent router classifies the query as <code>rag</code> (not greeting/developer). 6. RAG service retrieves top-5 similar chunks from ChromaDB using BAAI/bge-small-en-v1.5 embeddings. 7. Groq LLaMA 3.1 generates a structured JSON response containing the <code>answer</code> , <code>found_in_context</code> , <code>officeNodeId</code> , <code>doorNodeId</code> , <code>intent</code> , and <code>matched_person</code> . 8. Backend appends the user and bot messages to the chat document (creating a new chat if none exists) and returns the response to the mobile app.

Extensions	4a. If no relevant chunks are found, the system returns <code>found_in_context: false, needs_internet: true</code> , and prompts the student to enable internet search. 7a. If the LLM response is not valid JSON, the raw text is wrapped as the answer with <code>found_in_context: true</code> by default.
Special Requirements	The system shall return a response within an acceptable time under normal load to avoid degrading the conversational experience.

2.2.2 Use Case UC-02: Voice-Based Query Submission

Table 2.2: Use Case — Voice-Based Query Submission

Field	Description
UC #	UC-02 — Ref: FR-03, FR-04
UC Name	Voice-Based Query Submission
Level	Detailed
Description	A student records a voice query in the mobile app. The system transcribes the audio using faster-whisper, processes the transcribed text through the same RAG pipeline as text queries, and returns both a text reply and a synthesized audio response generated via Microsoft EdgeTTS.
Actor(s)	Student
Stakeholders	Students, Department Administration
Preconditions	Student is authenticated. Device microphone permission is granted.
Success Guarantee	The student receives a transcribed query, a relevant text reply, and a playable audio response.
Main Success Scenario	1. Student taps the microphone button and records a voice message. 2. Mobile app uploads the audio file (multipart/form-data) along with <code>chatId</code> to the Node.js voice endpoint. 3. Backend retrieves the last four chat messages and forwards the audio plus history to the Python AI service. 4. AI service transcribes the audio using faster-whisper (base model, CPU, int8). 5. Intent router classifies the transcribed text. 6. If intent is <code>rag</code> , the RAG pipeline generates the response as in

	UC-01. 7. AI service converts the answer text to speech via EdgeTTS (en-US-JennyNeural) and saves the audio file under /static. 8. Backend saves both the transcribed user message and bot reply (with metadata and audio URL) to the chat document and returns the combined result.
Extensions	4a. If transcription returns empty text, the system returns an error indicating empty transcription. 5a. If intent is <code>greeting</code> or <code>developer</code> , a canned response is returned without invoking the RAG pipeline, and TTS is still applied to that response.
Special Requirements	Transcription accuracy must be acceptable for standard spoken English queries in a normal acoustic environment.

2.2.3 Use Case UC-03: Indoor Navigation with Pathfinding

Table 2.3: Use Case — Indoor Navigation with Pathfinding

Field	Description
UC #	UC-03 — Ref: FR-07
UC Name	Indoor Navigation with Pathfinding
Level	Detailed
Description	A student selects a starting room and a destination room on the interactive SVG floor map. The system computes the shortest path using a breadth-first search over the room-node-connection graph and renders the route on the map.
Actor(s)	Student
Stakeholders	Students
Preconditions	Student has navigated to the Location tab. The room and door-node graph (<code>ROOMS</code> , <code>DOOR_NODES</code> , <code>RAW_CONNECTIONS</code>) is loaded into the app.
Success Guarantee	The student sees a highlighted route from the selected start room to the destination room on the indoor map.
Main Success Scenario	1. Student opens the Location tab, which renders <code>IndoorDeptMap</code> with the department floor plan. 2. Student selects a start room via the <code>BottomGestureBar</code> . 3. Student selects a destination room. 4. The app

	converts both room IDs to their corresponding <code>doorNodeIds</code> . 5. <code>makeConnectionsBidirectional()</code> builds an undirected adjacency graph from <code>RAW_CONNECTIONS</code> . 6. <code>findPath()</code> performs a breadth-first search from the start door node to the destination door node. 7. <code>getPathCoords()</code> maps the resulting node sequence to coordinates, which are drawn as a route overlay on the SVG map.
Extensions	2a. Navigation may also be triggered automatically when the AI chatbot detects a visit intent and supplies an <code>officeNodeId/doorNodeId</code> (see UC-01). 6a. If no path exists between the two nodes, <code>findPath()</code> returns an empty array and no route is drawn.
Special Requirements	The pathfinding computation shall complete instantly (client-side) without a network round trip.

2.2.4 Use Case UC-04: QR Code Scanning for Self-Location

Table 2.4: Use Case — QR Code Scanning for Self-Location

Field	Description
UC #	UC-04 — Ref: FR-08, FR-13
UC Name	QR Code Scanning for Self-Location
Level	Detailed
Description	A student scans a QR code physically placed outside a department room using the mobile app's camera. The scanned code identifies the room, and the app automatically sets it as the student's current location on the indoor navigation map.
Actor(s)	Student
Stakeholders	Students, Department Administration
Preconditions	The room has a QR code generated and printed by an administrator (UC-08). Camera permission is granted.
Success Guarantee	The scanned room is set as the student's current location, ready for destination selection and route computation.

Main Success Scenario	1. Student opens the QR tab and scans the QR code outside a room using <code>QrScanner</code> . 2. The QR payload (containing <code>nodeId</code> and <code>intent=from</code> , or location metadata) is parsed. 3. The app navigates to the Location screen with <code>nodeId</code> and <code>intent</code> as route parameters. 4. The Location screen resolves the <code>doorNodeId</code> to a room object using <code>ROOMS</code> . 5. If <code>intent === "from"</code> , the resolved room is set as <code>startRoom</code> ; otherwise it is set as <code>endRoom</code> . 6. The student selects the remaining endpoint, and the system computes and displays the route (UC-03).
Extensions	4a. If the scanned <code>nodeId</code> does not match any known room, the scan is ignored and no location is set.
Special Requirements	QR codes shall be scannable under normal indoor lighting conditions using the device's rear camera.

2.2.5 Use Case UC-05: User Registration and Login

Table 2.5: Use Case — User Registration and Login

Field	Description
UC #	UC-05 — Ref: FR-01
UC Name	User Registration and Login
Level	Detailed
Description	A student logs in using their registration number, or an administrator logs in using their email, both with a password. On success, the system issues a JWT token and persists the session on the device.
Actor(s)	Student, Administrator
Stakeholders	Students, Department Administration
Preconditions	The user account already exists in MongoDB (students self-register or are created by an admin; admins are created via <code>register-admin</code>).
Success Guarantee	A valid JWT token and user profile are returned and stored locally, granting access to role-appropriate screens.
Main Success Scenario	1. User enters either a registration number (student) or email (admin) plus a password on the login screen. 2. The mobile app or admin panel

	sends a POST request to <code>/api/auth/login</code> . 3. The backend looks up the user by <code>registrationNumber</code> (role must be <code>student</code>) or by <code>email</code> (role must be <code>admin</code>). 4. The backend compares the submitted password against the bcrypt hash using <code>comparePassword()</code> . 5. On match, <code>generateToken()</code> issues a signed JWT containing the user ID and role. 6. The backend returns the token and a sanitized user object (excluding the password). 7. The client stores the token and user object locally and navigates to the home/dashboard screen based on role.
Extensions	3a. If no user is found, or the role does not match the login method used, the system returns "Invalid credentials". 4a. If the password does not match, the system returns "Invalid credentials".
Special Requirements	Passwords shall be stored as bcrypt hashes (cost factor 10) and never returned in API responses.

2.2.6 Use Case UC-06: Knowledge Document Upload and RAG Indexing

Table 2.6: Use Case — Knowledge Document Upload and RAG Indexing

Field	Description
UC #	UC-06 — Ref: FR-11
UC Name	Knowledge Document Upload and RAG Indexing
Level	Detailed
Description	An administrator uploads a departmental document (PDF, DOCX, XLSX, CSV, or TXT) through the admin panel. The system stores the file in Cloudinary, records its metadata in MongoDB, and forwards it to the Python AI service for chunking, embedding, and indexing into ChromaDB so it becomes immediately retrievable by the RAG pipeline.
Actor(s)	Administrator
Stakeholders	Department Administration, Students (as downstream beneficiaries)
Preconditions	Administrator is authenticated with an <code>admin</code> role.

Success Guarantee	The document is stored in Cloudinary, its metadata is saved in the DATA collection, and its content is searchable through the AI chatbot.
Main Success Scenario	1. Administrator selects a file and provides a file name and description in the admin panel. 2. Admin panel sends a multipart POST request to /api/data/add-data. 3. Backend checks for a duplicate file name in the DATA collection. 4. Backend uploads the file buffer to Cloudinary (virtual_helpdesk_files, resource type raw). 5. Backend creates a DATA document referencing the Cloudinary URL and public ID. 6. Backend forwards the same file as form-data to the Python /documents/upload endpoint. 7. The AI service reads the file based on extension (read_file), splits it into chunks (RecursiveCharacterTextSplitter, 600/150), normalizes each chunk's text, generates embeddings via BAAI/bge-small-en-v1.5, and stores them in the ChromaDB university_docs collection. 8. The AI service returns the chunk count, and the backend responds with the created DATA record.
Extensions	3a. If a file with the same name already exists, the request is rejected with a 409 conflict. 6a. If the Python service fails, the backend rolls back by deleting the Cloudinary file and the MongoDB record, returning a 500 error.
Special Requirements	Supported file types are PDF, DOCX, XLSX/XLS, CSV, and TXT; unsupported types raise an error during reading.

2.2.7 Use Case UC-07: FAQ Management

Table 2.7: Use Case — FAQ Management

Field	Description
UC #	UC-07 — Ref: FR-12
UC Name	FAQ Management
Level	Detailed

Description	An administrator creates, updates, deletes, and toggles the status of FAQ entries organized by category (general, technical, fee, admissions, attendance, results, and others) through the admin panel.
Actor(s)	Administrator
Stakeholders	Department Administration, Students
Preconditions	Administrator is authenticated with an <code>admin</code> role.
Success Guarantee	The FAQ collection in MongoDB accurately reflects the administrator's changes and is available for filtering and search.
Main Success Scenario	1. Administrator navigates to the FAQ management section and selects "Add FAQ". 2. Administrator enters a question, answer, category, and status. 3. Admin panel sends a POST request to <code>/api/faq/add-faq</code> . 4. Backend checks for a duplicate question (case-sensitive exact match after trimming). 5. Backend creates the FAQ document with <code>createdById</code> and <code>createdByName</code> set to the requesting admin. 6. For updates, the admin panel sends a PUT request to <code>/api/faq/update-faq/:id</code> with only the changed fields (question, answer, category, status), which the backend merges into the existing document. 7. For listing, the backend supports pagination, search (question/answer regex), and filtering by category and status, returning paginated results plus aggregate statistics.
Extensions	4a. If a duplicate question exists, the system returns a 400 error. 6a. If no valid fields are provided for update, the system returns a 400 error.
Special Requirements	FAQ answers are limited to 2000 characters; questions must be unique across the collection.

2.2.8 Use Case UC-08: Location and QR Code Management

Table 2.8: Use Case — Location and QR Code Management

Field	Description
UC #	UC-08 — Ref: FR-13
UC Name	Location and QR Code Management
Level	Detailed

Description	An administrator adds a department room as a location record, including its image, category, building/floor metadata, map position, and route points. On save, the system automatically generates a QR code encoding the location's identity and uploads both the image and QR code to Cloudinary.
Actor(s)	Administrator
Stakeholders	Department Administration, Students
Preconditions	Administrator is authenticated with an <code>admin</code> role and has prepared a location image.
Success Guarantee	A new <code>LOCATION</code> document exists with a valid image URL and an auto-generated, uploaded QR code that encodes the location ID, name, building, and floor.
Main Success Scenario	1. Administrator fills in location details (name, description, category, status, building, floor, map position, route points) and selects an image in <code>DashAddLocation</code>. 2. Admin panel sends a multipart POST request to <code>/api/location/add-location</code>. 3. Backend checks for a duplicate location name (case-insensitive). 4. Backend uploads the location image to the cloud based asset management platform i.e. Cloudinary (<code>virtual_helpdesk_files/locations/location_images</code>). 5. Backend creates the <code>LOCATION</code> document with temporary placeholder QR fields. 6. Backend serializes <code>{location_id, location_name, building, floor}</code> as JSON and generates a QR code data URL via the <code>qrcode</code> library. 7. Backend converts the QR data URL to a buffer and uploads it to Cloudinary (<code>.../locations/qr_codes</code>). 8. Backend updates the <code>LOCATION</code> document with the QR code URL and public ID and saves it.
Extensions	3a. If a location with the same name exists, the request is rejected with a 409 conflict. Error path: if any later step fails, the controller cleans up the uploaded image, the uploaded QR code, and the partially created database document.
Special Requirements	Each location's QR code shall remain valid for scanning by the mobile app's QR scanner (UC-04) as long as the location is not deleted.

2.2.9 Use Case UC-09: Admin Dashboard Statistics Viewing

Table 2.9: Use Case — Admin Dashboard Statistics Viewing

Field	Description
UC #	UC-09 — Ref: FR-14
UC Name	Admin Dashboard Statistics Viewing
Level	Detailed
Description	Upon login, an administrator views a dashboard summarizing key operational metrics: total users, total students, total uploaded knowledge documents, total FAQs, and total active FAQs.
Actor(s)	Administrator
Stakeholders	Department Administration
Preconditions	Administrator is authenticated with an <code>admin</code> role.
Success Guarantee	The dashboard displays counts that accurately reflect the current state of the <code>User</code> , <code>DATA</code> , and <code>FAQ</code> collections.
Main Success Scenario	1. Administrator logs in and is routed to the dashboard. 2. Admin panel sends a <code>GET</code> request to <code>/api/dashboard/dashboard-data</code> . 3. Backend executes <code>countDocuments()</code> queries: total users, total students (<code>role: "student"</code>), total files in <code>DATA</code> , total FAQs, and total active FAQs (<code>status: "active"</code>). 4. Backend returns the aggregated <code>stats</code> object. 5. Admin panel renders the statistics in dashboard cards.
Extensions	None identified.
Special Requirements	This endpoint is restricted to users with the <code>admin</code> role via the <code>authorize("admin")</code> middleware.

2.3 System Sequence Diagrams

The following System Sequence Diagrams (SSDs) illustrate the message-level interactions between an actor, the system's interfaces, and its underlying services for the most significant use cases described above. Each SSD treats the system as a black box from the actor's perspective while showing the internal hops across the mobile/admin client, the Node.js backend, the Python AI service, MongoDB, ChromaDB, and external LLM providers needed to fulfil the request.

2.3.1 SSD — Natural Language Query Submission (UC-01)

This SSD traces a text query from the mobile app through the Node.js backend, which retrieves recent chat history before delegating to the Python AI service. The AI service detects intent, retrieves context from ChromaDB, and calls Groq's LLaMA 3.1 model to produce a structured JSON response, which is persisted to the chat document and returned to the user.

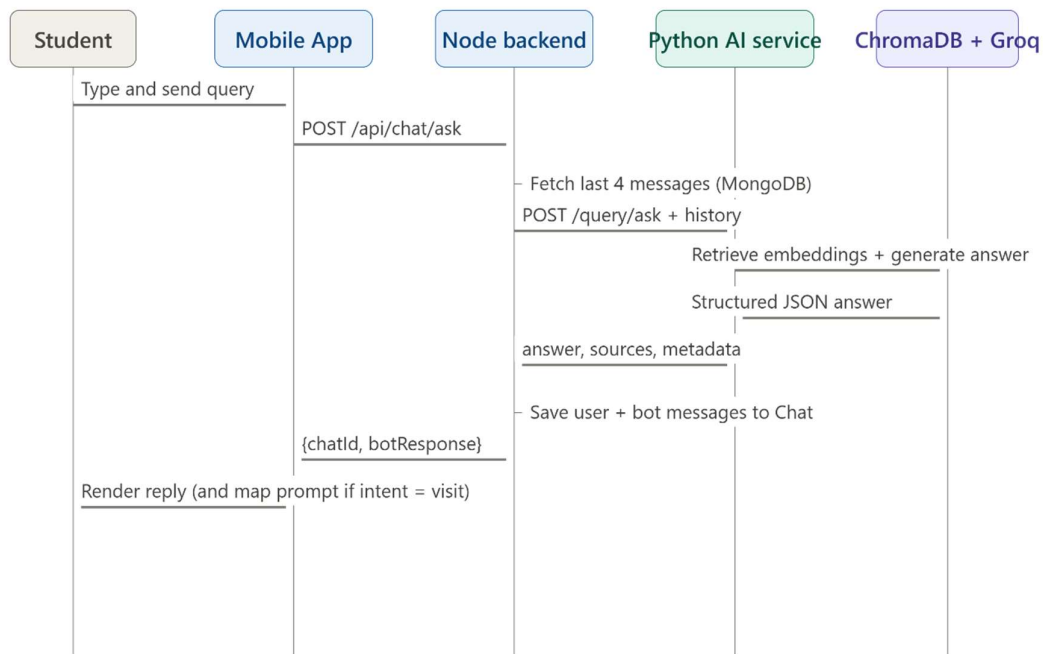


Figure 2.2: SSD — Natural Language Query Submission

2.3.2 SSD — Voice-Based Query Submission (UC-02)

This SSD shows a voice query flowing from the mobile app's audio recorder through the Node backend to the Python AI service, where faster-whisper transcribes the audio before the same RAG pipeline as the text flow runs, and EdgeTTS converts the final answer into a playable audio response.

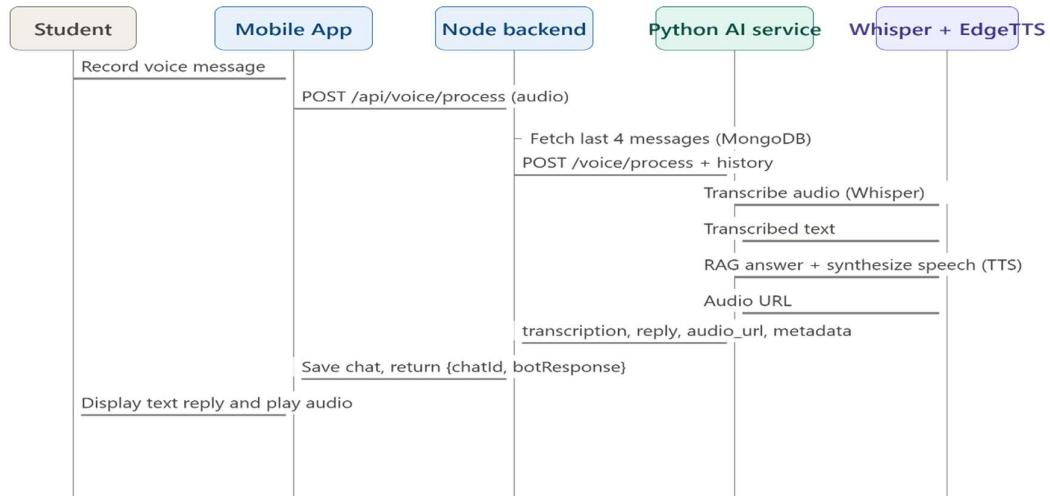


Figure 2.3: SSD — Voice-Based Query Submission

2.3.3 SSD — User Registration and Login (UC-05)

This SSD shows a user submitting login credentials (registration number for students, email for admins) from the client to the Node backend, which queries MongoDB for the matching account, validates the password hash, issues a JWT, and returns the token and profile to the client for local storage.

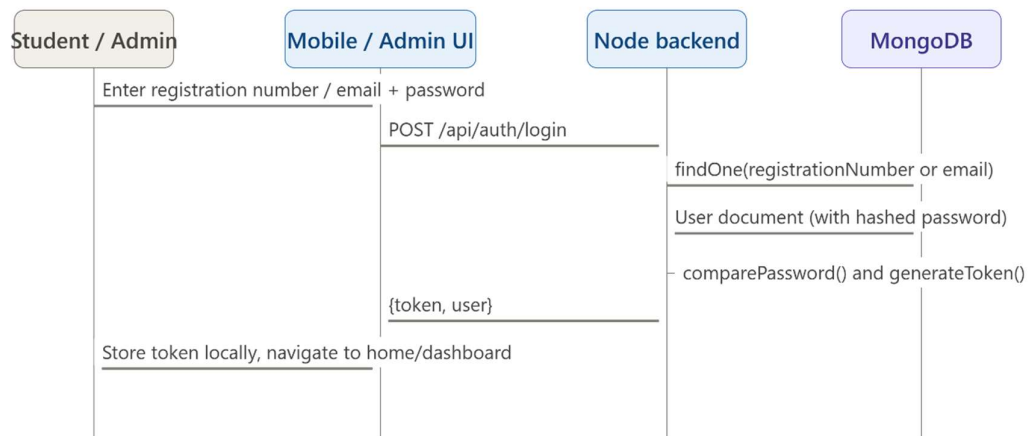


Figure 2.4: SSD — User Registration and Login

2.3.4 SSD — Knowledge Document Upload and RAG Indexing (UC-06)

This SSD shows an administrator uploading a document from the admin panel. The Node backend uploads the file to Cloudinary, persists its metadata in MongoDB, and forwards the same file to the Python AI service, which chunks, embeds, and indexes the content into ChromaDB, completing the document's availability to the RAG pipeline.

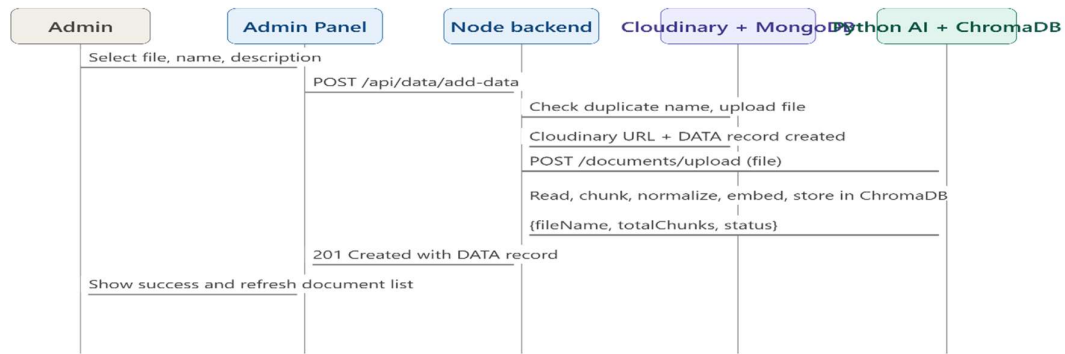


Figure 2.5: SSD — Knowledge Document Upload and RAG Indexing

2.3.5 SSD — Location and QR Code Management (UC-08)

This SSD shows an administrator creating a new location record. The Node backend uploads the location image to Cloudinary, persists the location, generates a QR code encoding the location's identity, uploads the QR image to Cloudinary, and updates the location record with the final QR reference.

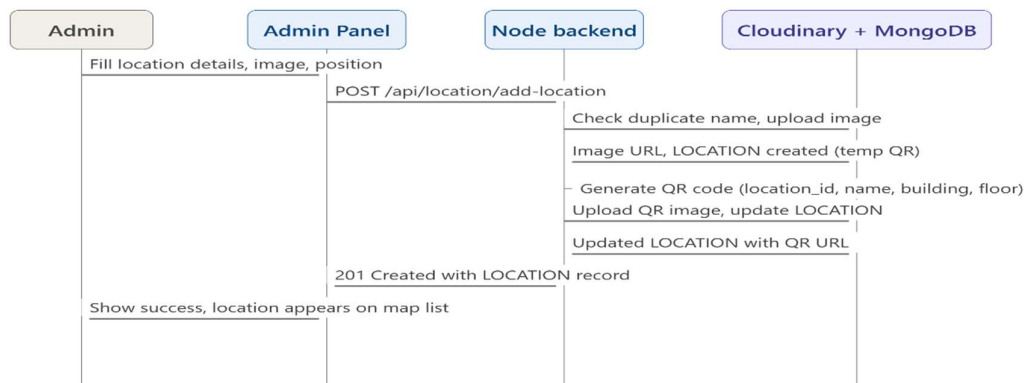


Figure: 2.6: SSD — Location and QR Code Management

2.4 Domain Model

The Domain Model represents the key conceptual entities of the AI-Based Virtual Helpdesk system and the relationships among them, independent of their implementation as Mongoose schemas, REST payloads, or ChromaDB collections. It establishes a shared vocabulary — Users, Chats, Messages, Knowledge Documents, FAQs, and Locations — that recurs throughout the requirements, use cases, and design chapters of this report. Each conceptual entity identified here is later refined into a concrete database schema and class structure in the design chapters, making this model the conceptual bridge between the requirements analysis (Chapter 2) and the system design (Chapter 3).

At the centre of the model is the User entity, which represents anyone who interacts with the system — a Student or an Administrator — distinguished by a role attribute and carrying role-specific information such as registration number and department. A User initiates Chat sessions, manages Locations, uploads knowledge DATA documents, and authors FAQ entries, making User the originating entity for almost all other activity in the system.

Primary entities identified in the Domain Model include:

- **User** (userId, name, email, passwordHash, role, registrationNumber, department, profilePicture, createdAt) — represents a student or administrator account and their authentication credentials.
- **Chat** (chatId, userId, title, createdAt, updatedAt) — represents one ongoing conversation session between a student and the AI assistant.
- **Message** (messageId, chatId, sender, text, timestamp) — represents a single user or bot message within a Chat, ordered chronologically.
- **Metadata** (sourceDocuments, intent, matchedPerson, officeNodeId, doorNodeId, audioUrl) — represents AI-generated provenance and routing information attached to a bot Message, linking it back to the knowledge base and, where applicable, to the indoor navigation graph.
- **DATA** (dataId, fileName, fileLink, publicId, description, uploadedById, uploadedByName, createdAt) — represents a departmental knowledge document (handbook, fee structure, notice, etc.) uploaded by an administrator.
- **KnowledgeChunk** (chunkId, text, sourceFile, chunkIndex, embedding) — represents a normalized, embedded segment of a DATA document, stored in the vector database and retrieved during RAG-based query answering.
- **FAQ** (faqId, question, answer, category, status, createdById, createdByName, createdAt, updatedAt) — represents a frequently asked question and its verified answer, organized by category.
- **Location** (locationId, locationName, description, category, status, building, floor, image, createdById) — represents a physical room, office, or facility within the IT department.

- **QRCode** (qrUrl, publicId) — represents the scannable code generated for a Location, encoding its identity for the mobile app's QR scanning feature.
- **Position** (x, y) — represents the coordinates of a Location on the indoor SVG floor map.
- **RoutePoint** (x, y) — represents an intermediate waypoint used by the pathfinding algorithm to draw a navigable route to or from a Location.

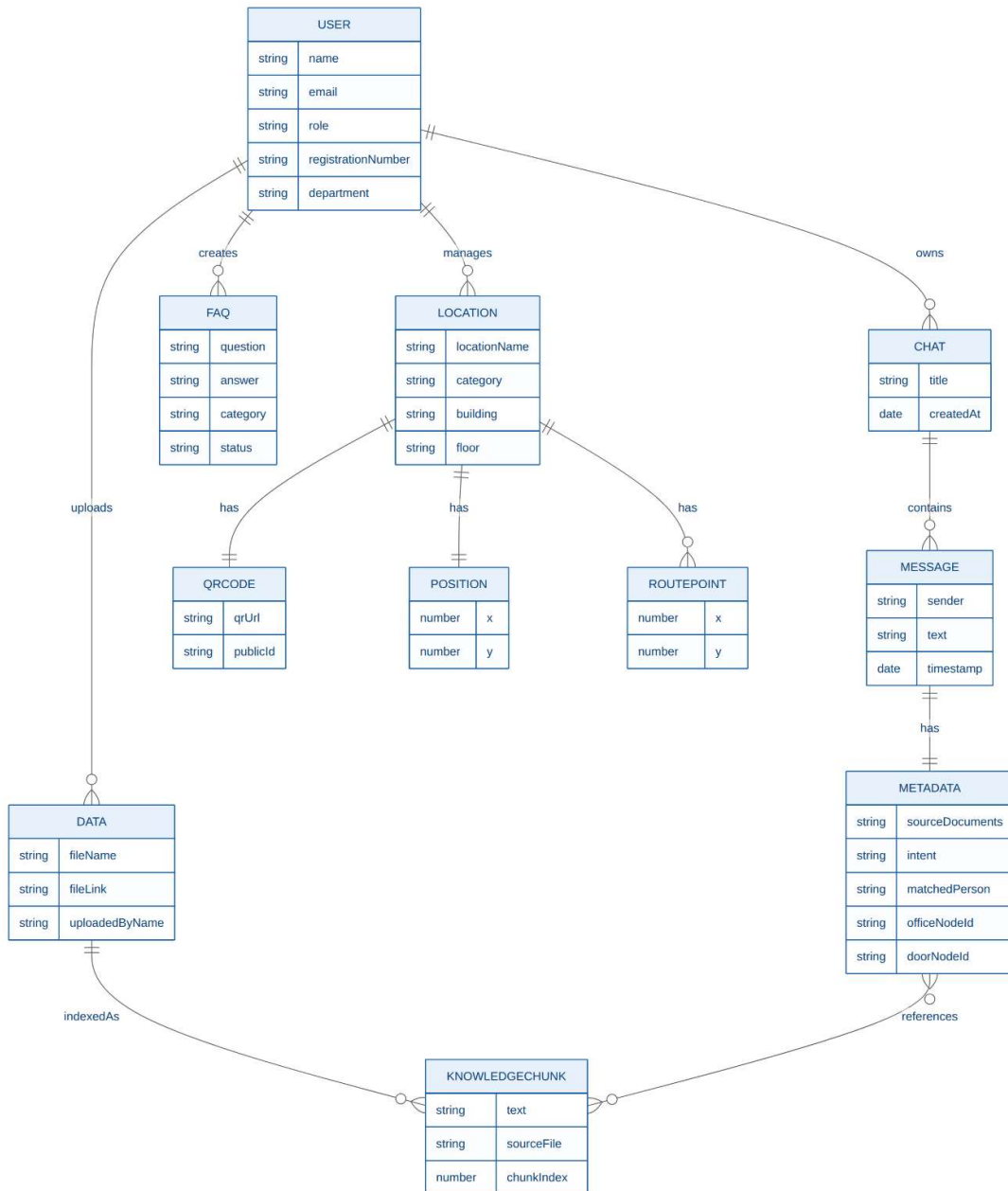


Figure 2.7: Domain Model - ERD

CHAPTER 3

PROJECT DESIGN

The project design chapter translates the requirements and analysis artefacts established in the preceding chapters into concrete architectural and structural decisions that guided the implementation of the AI-Based Virtual Helpdesk system. Whereas Chapter 1 captured what the system must do and Chapter 2 modelled the interactions between actors and the system, this chapter addresses how the system is internally organised, how its components relate to one another, how its data is structured and persisted, and how the system's layers communicate with each other.

The design decisions documented here were made with the goals of modularity, maintainability, scalability, and alignment with the actual implemented codebase, ensuring that every element described in this chapter reflects the working system as deployed.

3.1 Design Class Diagram

The Design Class Diagram extends the domain model by introducing implementation-level details including attributes with their data types, method signatures, visibility modifiers, and inter-class relationships. It serves as the structural blueprint of the system's object-oriented organisation, mapping directly to the implemented codebase across the Node.js backend, the Python AI service, and the React Native mobile application.

The primary classes identified in the system are as follows:

- **User** — the central class representing both students and administrators. It holds identity attributes (`userId`, `name`, `email`, `passwordHash`), role-based attributes (`role`, `registrationNumber`, `department`, `semester`, `program` for students; `designation` for admins), and profile image data. Key methods include `login()` which returns a signed JWT, and `generateOTP()` which produces a SHA-256 hashed one-time password for password recovery.
- **Chat** — represents a single conversation session belonging to a **User**. It holds the `chatId`, a reference to `userId`, a title auto-generated from the first query, and an embedded array of **Messages**. Methods include `getMessages()`, `addMessage()`, `getRecent()`, and `getById()`.
- **Message** — represents a single turn within a **Chat**, with sender (user or bot), text content, and a timestamp. Each bot **Message** carries an associated **Metadata** object containing AI-generated provenance data.
- **Metadata** — an embedded value object attached to each bot **Message**, holding `sourceDocuments`, `officeNodeId`, `doorNodeId`, `intent`, `matchedPerson`, and `audioUrl`. These fields are populated by the RAG pipeline and navigation service and are used by the mobile app to trigger map navigation automatically.

- **DATA (KnowledgeDoc)** — represents an uploaded departmental document. It stores the `fileName`, `Cloudinary fileLink`, `cloudinaryPublicId`, and a reference to the uploading administrator. On upload, the document is forwarded to the AI service which chunks, embeds, and indexes it into ChromaDB.
- **RAGService** — the core AI processing class in the Python FastAPI service. It encapsulates the ChromaDB vector store, the BAAI/bge-small-en-v1.5 embedding model, and the LLaMA 3.1 8B LLM accessed via Groq. Key methods include `ragAnswer()`, `retrieveDocs()`, `buildContext()`, `intentDetect()`, and `normalize()`.
- **VoiceService** — handles all speech processing, encapsulating the faster-whisper Whisper model for speech-to-text transcription and Microsoft EdgeTTS (en-US-JennyNeural voice) for text-to-speech synthesis. Its `transcribe()` method saves an uploaded audio file, runs Whisper inference, and returns the transcribed text; `textToSpeech()` generates an MP3 file and returns its static URL.
- **FAQ** — represents a structured frequently asked question, with question, answer, category (from a fixed enumeration), status, and creator references. Methods include CRUD operations managed by the admin panel via the Node.js REST API.
- **Location** — represents a physical room or office in the IT Department. It stores `locationName`, `category`, `building`, `floor`, `position` (x, y coordinates on the SVG map), `routePoints`, and the QR code URL. The `generateQR()` method produces a unique QR code via the `qrcode` library, uploads it to Cloudinary, and returns the hosted URL.
- **InternetService** — the fallback AI service class, encapsulating the Tavily search client and two LLMs (Google Gemini 2.5 Flash and LLaMA 3.1 via Groq). Its `chooseLLM()` method selects between Groq and Gemini based on context token length and query complexity.
- **NavigationService** — encapsulates the indoor pathfinding logic. It holds the room nodes array and adjacency graph, and implements BFS-based `findPath()` and `getCoords()` methods to compute and render navigation routes on the SVG floor map.

The following diagram represents the complete class diagram with all attributes, methods, and relationships including associations, compositions, and dependencies.

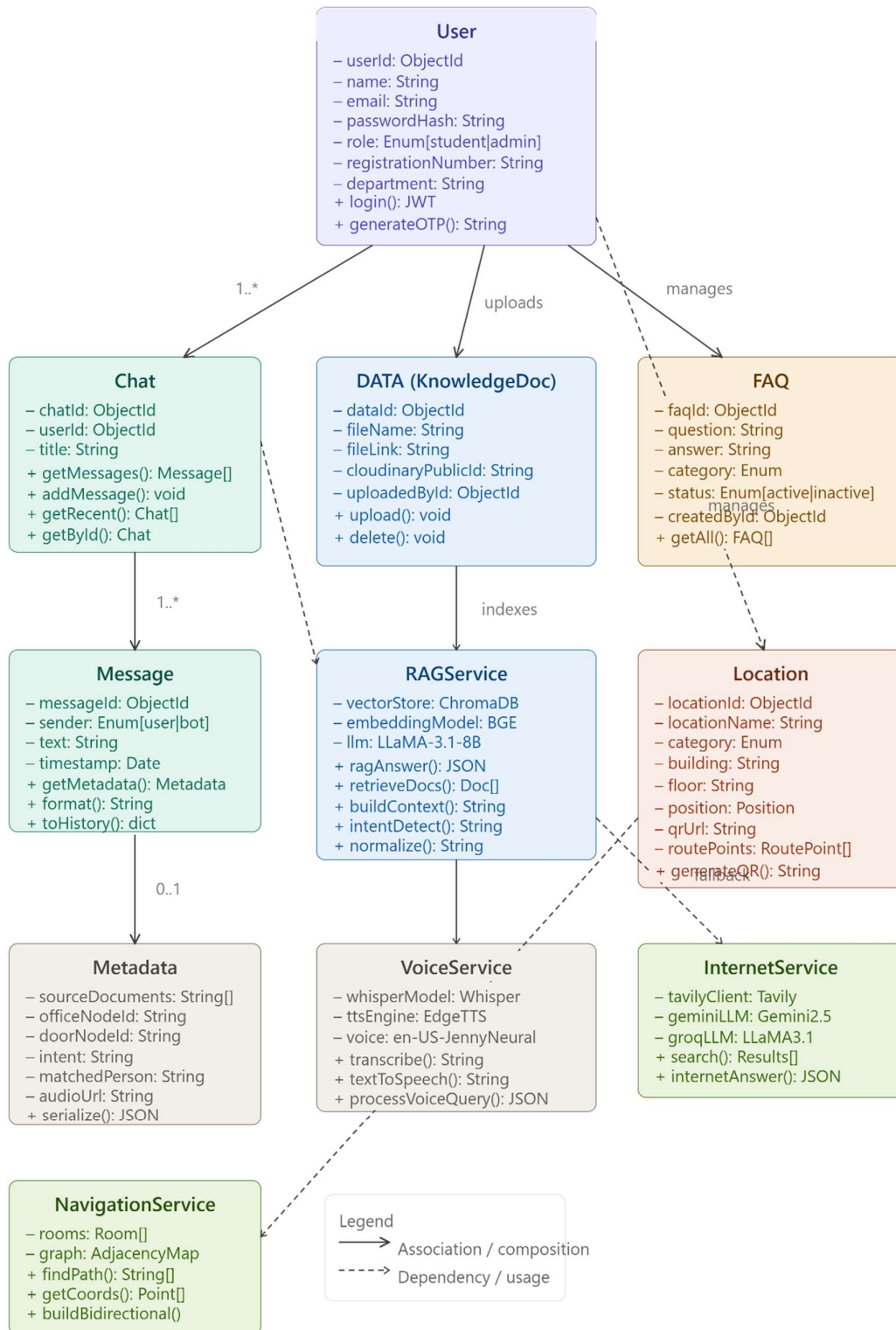


Figure 3.1: Class Diagram

3.2 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram for the AI-Based Virtual Helpdesk captures all persistent data entities stored in MongoDB and their structural relationships. As this system uses a document-oriented NoSQL database (MongoDB Atlas), relationships are represented as references (ObjectId foreign keys) rather than traditional relational foreign-key constraints. The ERD reflects the exact schema implemented in the five Mongoose models of the Node.js backend.

The primary entities and their relationships are described below:

- **User** is the root entity of the system. It maintains a one-to-many relationship with the **Chat** entity, as a single user can initiate multiple conversation sessions. The **User** entity also has a created-by relationship with **FAQ** and **Location** records, where each **FAQ** and **Location** stores the ObjectId and name of the administrator who created it.
- **Chat** holds an embedded one-to-many relationship with the **Message** entity. In the implemented schema, Messages are stored as a sub-document array within each Chat document rather than as a separate collection, reflecting MongoDB's document embedding pattern.
- **Message** carries an embedded **Metadata** object as part of its schema. The **Metadata** sub-document is populated only for bot messages and contains source document references, navigation node identifiers, and AI-extracted intent information.
- **DATA** represents uploaded knowledge documents and maintains a many-to-one relationship with **User** (each document is uploaded by one admin). The **DATA** entity has a logical dependency on the **RAG pipeline** — each uploaded document is chunked and stored in ChromaDB as **KnowledgeChunks**, which are not persisted in MongoDB but in the vector database.
- **FAQ** maintains a many-to-one relationship with **User** (each FAQ is created by one admin) and is independently managed without direct embedding in other documents.
- **Location** maintains a many-to-one relationship with **User** (created and updated by admins) and contains embedded **Position** (x, y coordinates) and **RoutePoint** arrays as nested sub-documents within its schema.

The following ERD diagram represents the complete entity-relationship structure with cardinalities and attribute listings for all entities described above.

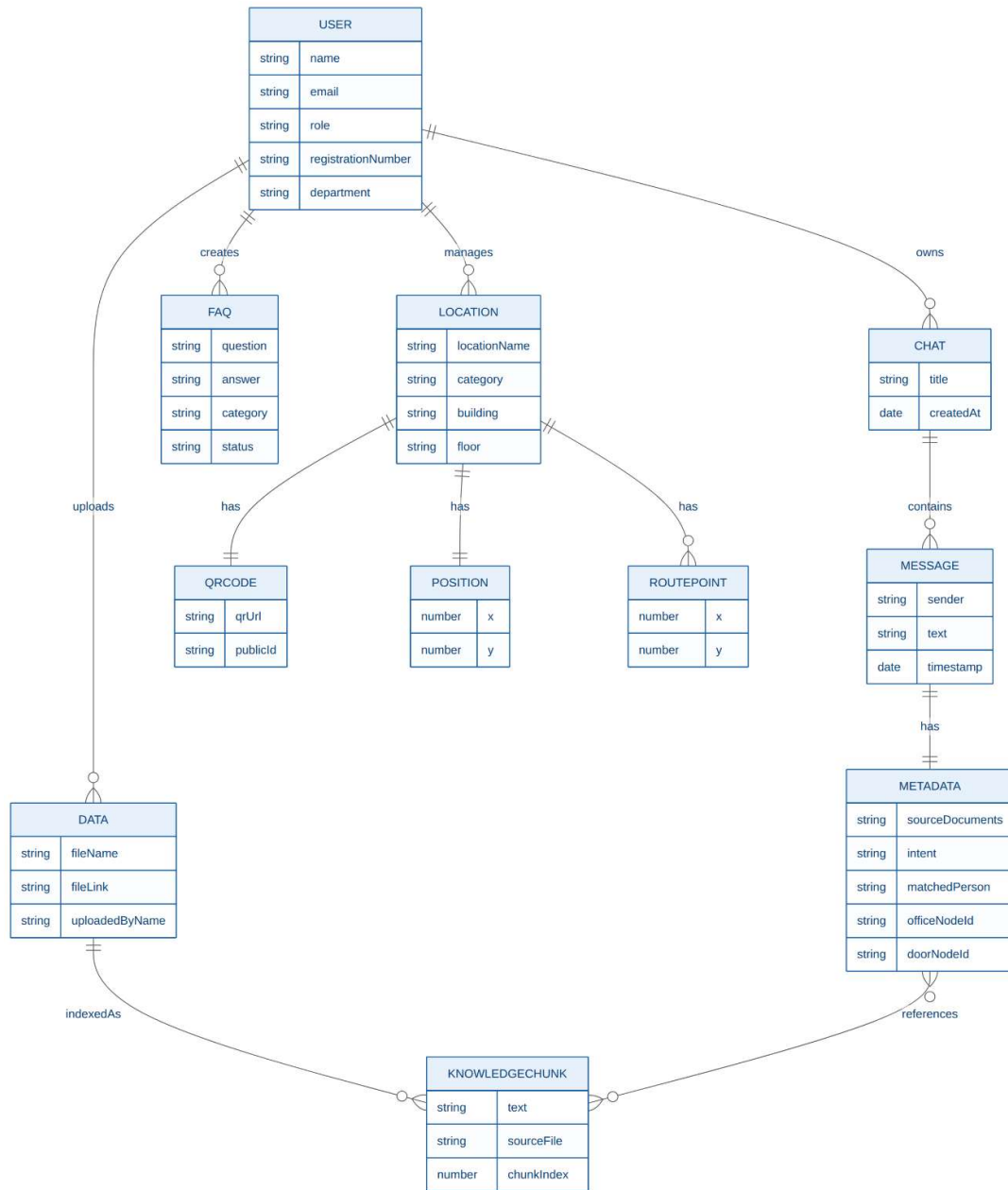


Figure 3.2: Entity Relationship Diagram (ERD)

3.3 System Architecture Diagram

The AI-Based Virtual Helpdesk follows a 5-Layer Architecture pattern. This architectural style was selected over a simpler 3-tier model because the system's AI processing responsibilities are substantial enough to warrant a dedicated, independently deployable service tier, and because the system integrates with a significant number of external third-party services that must be isolated from the core business logic layer. The five layers are: the Presentation Layer, the API Gateway and Business Logic Layer, the AI Processing Layer, the Data and Storage Layer, and the External Services Layer. All three backend services (Node.js server, Python AI service, React admin panel) are containerised using Docker, enabling consistent and reproducible deployment.

The key architectural characteristics of the system are as follows:

- The **Presentation Layer** consists of two independent client applications: the React Native/Expo mobile application used by students, and the React/Vite admin web panel used by department administrators. Both communicate exclusively with the Node.js backend via RESTful HTTPS APIs.
- The **API Gateway and Business Logic Layer** is implemented as a Node.js/Express server running on Port 5000. It handles all authentication (JWT middleware), request validation (express-validator), file upload processing (Multer), email notifications (Nodemailer), and QR code generation (qrcode library). It acts as the single entry point for all client requests and orchestrates calls to the Python AI service.
- The **AI Processing Layer** is an independently deployed Python FastAPI service running on Port 8000. It hosts the complete RAG pipeline (LangChain orchestration, ChromaDB vector retrieval, LLaMA 3.1 8B inference via Groq API), the intent routing module (using sentence-transformers MiniLM-L6-v2 for semantic classification), the text intelligence module (fuzzy and phonetic name normalization using rapidfuzz and pyphonetics), and the voice processing pipeline (faster-whisper for STT and Microsoft EdgeTTS for TTS).
- The **Data and Storage Layer** comprises three distinct storage systems: MongoDB Atlas (M0 free cluster) for structured document storage of users, chats, FAQs, locations, and knowledge document metadata; ChromaDB (persistent local client) for storing vector embeddings of all indexed knowledge documents in the `university_docs` collection; and Cloudinary (free tier) for hosting location images, QR code images, and user profile pictures.
- The **External Services Layer** encapsulates all third-party AI APIs and cloud services consumed by the system, including the Groq API (for LLaMA 3.1 8B inference), Google Gemini 2.5 Flash API (for internet fallback complex queries), Tavily Search API (for web search in the internet fallback), Microsoft EdgeTTS cloud service (for neural text-to-speech), the local faster-whisper model (for on-device speech-to-text at zero API cost), and Nodemailer with Gmail SMTP (for OTP email delivery).

5-Layer System Architecture

AI-Based Virtual Helpdesk — GCUF IT Department

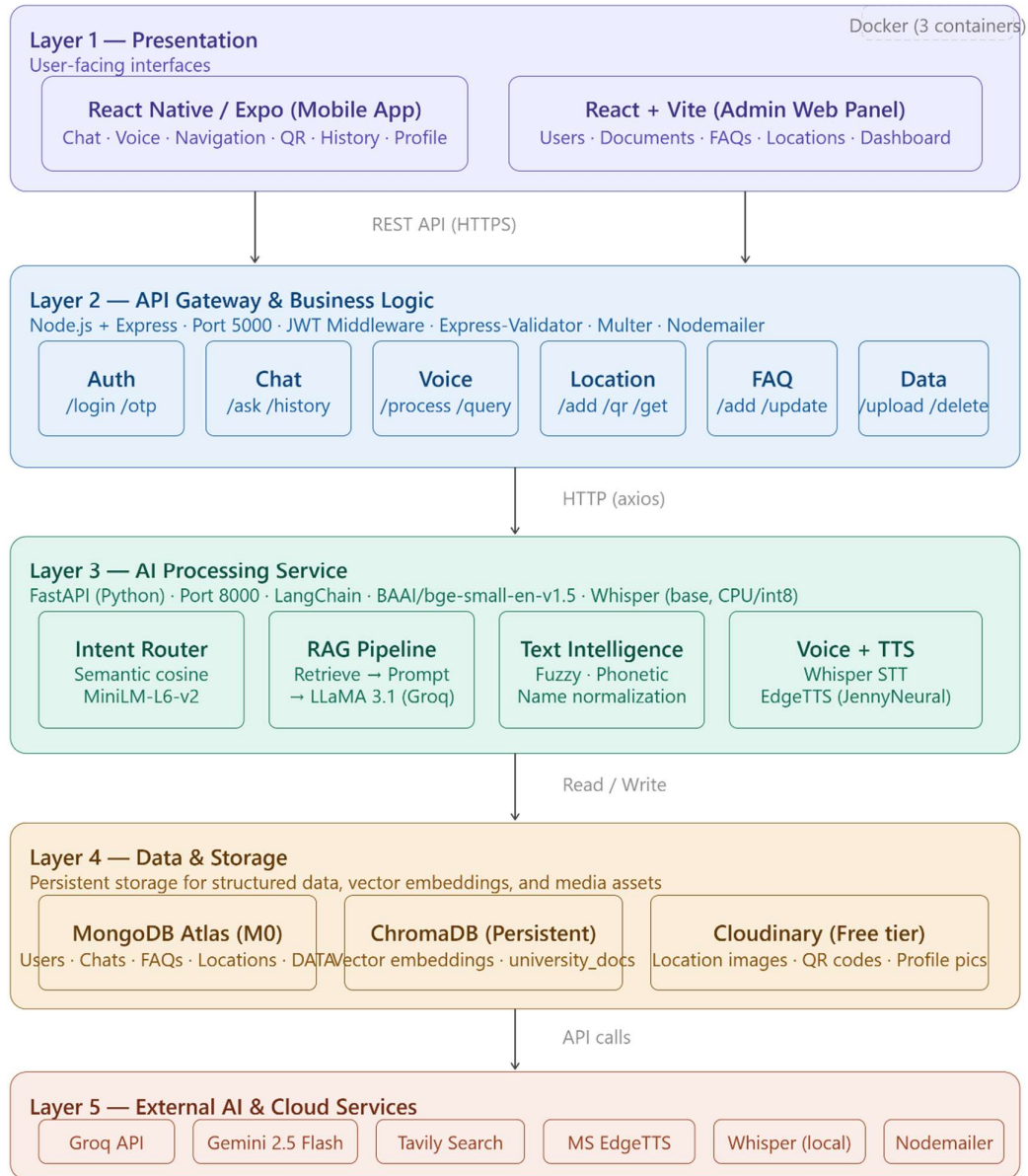


Figure 3.3: System Architecture Diagram

3.4 Database Design Details

The AI-Based Virtual Helpdesk employs a dual-database architecture combining MongoDB Atlas as the primary document database for all structured application data, ChromaDB as the vector database for the RAG knowledge base, and Cloudinary as the cloud-based binary asset store for images and QR codes.

MongoDB Atlas is the primary persistent store for all relational and document data. The database contains five collections, each corresponding to a Mongoose model defined in the Node.js backend. The User collection is the most foundational, storing both student and administrator accounts within a single collection distinguished by a role field. Student documents carry registration-specific fields such as registrationNumber, degreeType, degreeTitle, semester, program, and session, while administrator documents carry a designation field. All passwords are stored as bcrypt-hashed strings (10 salt rounds) and are never returned in query results due to the select: false directive. OTP fields (hashed OTP value, expiry timestamp, and attempt counter) are also embedded directly within each User document, enabling the password reset flow without a separate OTP collection. Profile image data is stored as a nested object containing the Cloudinary URL and public ID.

The **Chat** collection stores conversation sessions as top-level documents, each identified by a MongoDB ObjectId and linked to a User via a userId reference. Each Chat document embeds an array of Message sub-documents rather than using a separate collection. This embedding strategy was chosen because messages are never queried independently of their parent Chat and benefit from atomic reads. Each Message sub-document carries a sender field (user or bot), the message text, a timestamp, and — for bot messages — a Metadata nested object containing sourceDocuments (an array of source file names from ChromaDB), processing flags (found_in_context, needs_internet), navigation data (officeNodeId, doorNodeId), AI understanding fields (intent, matchedPerson), and the audioUrl for voice responses. The chat history retrieval endpoint slices the last four messages from the embedded array to inject as conversational memory into the RAG pipeline on each new query.

The **DATA** collection stores metadata for each knowledge document uploaded by administrators. It does not store the binary file content itself (which is stored on Cloudinary) but instead stores the fileName, fileLink (Cloudinary URL), cloudinaryPublicId (for deletion), fileDescription, and references to the uploading administrator. This design ensures that the Node.js backend maintains an authoritative record of all indexed documents, enabling administrators to view, manage, and delete documents through the admin panel while the actual vector embeddings reside in ChromaDB.

The **FAQ** collection stores structured frequently asked questions with a question, answer, category (from a controlled enumeration including general, technical, security, fee, admissions, attendance, result, courses, examination, rules, events, library, and scholarship), status (active or inactive), and creator references. A unique index on the question field prevents duplicate FAQ entries. The status field enables administrators to deactivate FAQs without deleting them, preserving audit history.

The **LOCATION** collection is the most structurally complex document schema in the system. Each Location document stores the location_name (unique),

location_description, location_category (classroom, office, lab, washroom, hall, etc.), status, building name, floor identifier, a nested position object containing x and y coordinates on the SVG floor map, an array of route_point objects (each with x and y coordinates defining intermediate waypoints for the pathfinding algorithm), the location_image URL (Cloudinary), the location_qr URL (Cloudinary-hosted QR code image), both Cloudinary public IDs for deletion, a location_views counter, and full creator and updater references. The position and route_points fields directly power the indoor navigation feature in the mobile app by providing the coordinate data used by the BFS pathfinding algorithm.

ChromaDB serves as the vector database for the RAG pipeline and operates as a persistent local client within the Python AI service container. All knowledge documents uploaded by administrators are processed through a pipeline that reads the file (supporting PDF, DOCX, and JSON formats via LangChain document loaders), splits the content into overlapping chunks using LangChain's RecursiveCharacterTextSplitter, generates 384-dimensional vector embeddings for each chunk using the BAAI/bge-small-en-v1.5 model from HuggingFace, and stores the chunks along with their embeddings and metadata (source file name, chunk index, Mongo document ID) into a single ChromaDB collection named university_docs. The collection uses cosine similarity as its distance metric (configured via the hnsw:space metadata parameter). At query time, the RAG pipeline encodes the normalized query into the same embedding space and retrieves the top five most semantically similar chunks using ChromaDB's similarity search, which are then injected as context into the LLM prompt.

Cloudinary functions as the binary asset management layer, handling three categories of assets: location room images uploaded by administrators, auto-generated QR code PNG images for each room location, and user profile pictures uploaded by students. All uploads are performed server-side via the Cloudinary Node.js SDK using stream-based uploads (via the streamifier library) so that files never need to be written to the server's local filesystem. Each asset is stored with a specific folder path (e.g., virtual_helpdesk_files/locations/location_images) and its Cloudinary public ID is persisted in MongoDB alongside the secure URL, enabling the system to delete assets from Cloudinary when the corresponding MongoDB document is deleted, ensuring no orphaned files accumulate in cloud storage.

The three database systems are designed to complement each other without overlap: MongoDB handles all structured, queryable, and relational application data; ChromaDB handles all unstructured semantic search over document content; and Cloudinary handles all binary media assets. Together they form a complete, coherent, and scalable data architecture for the system.

CHAPTER 4

IMPLEMENTATION

This chapter presents the implementation details of the AI-Based Virtual Helpdesk developed for the IT Department, Government College University Faisalabad (GCUF). It describes the practical realization of the system design presented in the previous chapter by explaining the major code segments, algorithms, and software components that collectively deliver the required functionality. The implementation follows a modular architecture consisting of a React Native mobile application, a Node.js/Express backend, a Python-based AI service utilizing Retrieval-Augmented Generation (RAG), and a React-based administrative panel. The following sections discuss the core implementation units, technologies employed, and system modules of the project.

4.1 Code/Algorithm of Main Scenarios and Main Units of the System

We will start with the **Authentication Module**, since every system operation depends on it.

4.1.1 JWT Authentication Token Generation

Description:

This code segment is responsible for generating a secure JSON Web Token (JWT) after a user successfully logs into the system. The generated token contains the user's unique identifier and role (student or administrator), allowing subsequent API requests to be authenticated without requiring repeated login operations. The token is digitally signed using a secret key stored in environment variables and expires automatically after a predefined duration, enhancing the overall security of the application.

Code Segment:

```
const generateToken = (userId, role) => {
  return jwt.sign(
    {
      id: userId,
      role,
    },
    process.env.JWT_SECRET,
    {
      expiresIn: process.env.JWT_EXPIRE,
    }
  );
};
```

Working:

The implementation performs the following operations:

- Receives the authenticated user's unique identifier and role.
- Creates a payload containing the user's information.
- Signs the payload using the application's secret key.
- Applies an expiration period defined in the environment configuration.
- Returns the generated JWT token to the client application.

Purpose:

- Secure user authentication
- Stateless session management
- Role-based authorization
- Protection of backend APIs

4.1.2 User Database Schema (MongoDB)

Description:

The User schema defines the structure of user documents stored within the MongoDB database. It supports both student and administrator accounts by maintaining personal information, authentication credentials, and role-specific attributes. Passwords are securely stored using bcrypt hashing, ensuring that sensitive information is never saved in plain text.

Code Segment:

```
const UserSchema = new mongoose.Schema({
  fullName: {
    type: String,
    required: true,
  },

  registrationNumber: {
    type: String,
    unique: true,
    sparse: true,
  },

  email: {
    type: String,
    unique: true,
    sparse: true,
  },

  password: {
    type: String,
    required: true,
  },

  role: {
    type: String,
    enum: ["student", "admin"],
    default: "student",
  },
});
```

Working:

This schema performs the following functions:

- Stores student and administrator information.
- Maintains unique registration numbers and email addresses.
- Stores encrypted passwords.
- Differentiates users through role-based access control.
- Serves as the primary authentication entity for the system.

Purpose:

- User data management
- Authentication support
- Authorization management
- Database consistency

4.1.3 User Login Controller

Description:

The login controller authenticates users by validating their credentials against stored database records. It supports registration-number-based login for students and email-based login for administrators. After successful verification of the bcrypt-hashed password, the controller generates a JWT token and returns authenticated user information while excluding sensitive fields.

Code Segment:

```
const user = await User.findOne(query);

if (!user) {
  return res.status(401).json({
    success: false,
    message: "Invalid credentials",
  });
}

const isMatch = await user.comparePassword(password);

if (!isMatch) {
  return res.status(401).json({
    success: false,
    message: "Invalid credentials",
  });
}

const token = generateToken(user._id, user.role);

res.json({
  success: true,
  token,
  user,
});
```

Working:

The controller executes the following sequence:

1. Searches for the user in MongoDB.
2. Verifies whether the account exists.
3. Compares the entered password with the stored bcrypt hash.
4. Generates a JWT authentication token.
5. Returns the authenticated user's information and token to the client.

Purpose:

- User authentication
- Password verification
- Secure session creation
- API authorization support

4.1.4 Chat Request Processing Controller

Description:

The Chat Request Processing Controller serves as the central entry point for all text-based interactions between students and the AI-powered virtual helpdesk. It receives the student's query from the mobile application, retrieves previous conversation history to maintain context, forwards the request to the Python AI service, and finally stores both the user's message and the generated response in the database. This controller acts as the communication bridge between the mobile frontend, Node.js backend, MongoDB database, and the AI service.

Code Segment:

```
const history = await getRecentMessages(chatId);

const aiResponse = await axios.post(
  `${AI_SERVICE_URL}/chat`,
  {
    query,
    history,
    use_internet,
  }
);

await saveChatMessage(chatId, query, aiResponse.data.answer);

return res.status(200).json(aiResponse.data);
```

Working:

The controller performs the following operations:

- Receives the student's text query.
- Retrieves recent conversation history for context preservation.
- Sends the request to the FastAPI AI service.
- Receives the AI-generated response.
- Stores both user and assistant messages in MongoDB.
- Returns the response to the mobile application.

Purpose:

- Centralized chat processing
- Backend-AI communication
- Chat history persistence
- Context-aware conversations

4.1.5 Multi-Turn Conversation Memory Retrieval

Description:

To support natural conversations, the system maintains short-term memory by retrieving the latest messages from the current chat session. Instead of treating every query independently, the previous interactions are supplied to the language model, enabling it to understand follow-up questions and maintain conversational continuity.

Code Segment:

```
const recentMessages = chat.messages
  .slice(-4)
  .map((message) => ({
    role: message.role,
    content: message.content,
  }));
```

Working:

The algorithm:

- Retrieves the current chat document.
- Selects the most recent four messages.
- Preserves both user and assistant roles.
- Formats them for AI service consumption.
- Supplies contextual information for subsequent queries.

Purpose:

- Context preservation

- Multi-turn conversations
- Improved AI understanding
- Better response quality

4.1.6 Query Normalization Algorithm

Description:

Students frequently enter faculty names with spelling mistakes, abbreviations, or phonetic variations. Before retrieval begins, the system preprocesses and normalizes the incoming query to improve semantic matching accuracy. This significantly increases the likelihood of retrieving the correct knowledge chunks from the vector database.

Code Segment:

```
normalized_query = normalize_text(query)

matched_person = match_faculty_name(
    normalized_query,
    faculty_dataset
)

processed_query = replace_with_canonical_name(
    normalized_query,
    matched_person
)
```

Working:

The normalization process:

- Cleans the input text.
- Removes unnecessary variations.
- Detects phonetic similarities.
- Matches misspelled faculty names.
- Replaces them with canonical names before retrieval.

Purpose:

- Improve retrieval accuracy
- Handle spelling mistakes
- Support natural language queries
- Increase chatbot reliability

4.1.7 Intent Classification Algorithm

Description:

Not every incoming message requires Retrieval-Augmented Generation. The system first analyzes the student's query to determine its intent and selects the most appropriate processing strategy. Greeting messages, developer-related queries, and departmental information requests are handled through different execution paths.

Code Segment:

```
intent = classify_intent(query)

if intent == "greeting":
    return greeting_response()

elif intent == "developer":
    return developer_response()

else:
    return rag_pipeline(query)
```

Working:

The algorithm:

- Receives the normalized query.
- Identifies the user's intent.
- Routes greeting messages directly.
- Routes developer information separately.
- Sends departmental queries to the RAG pipeline.

Purpose:

- Efficient request routing
- Reduce unnecessary AI processing
- Faster responses
- Modular architecture

4.1.8 RAG Document Retrieval Algorithm

Description:

The Retrieval-Augmented Generation (RAG) pipeline is the fundamental intelligence component of the Virtual Helpdesk. Instead of relying solely on the language model's internal knowledge, the system first retrieves semantically relevant document chunks from ChromaDB and provides them as context for response generation.

Code Segment:

```
embedding = embedding_model.embed_query(query)

results = collection.similarity_search(
    embedding,
    k=5
)

context = "\n".join(
    doc.page_content for doc in results
)
```

Working:

The retrieval process performs:

1. Generate an embedding vector for the student's query.
2. Search ChromaDB using semantic similarity.
3. Retrieve the top five relevant document chunks.
4. Merge retrieved chunks into a unified context.
5. Forward the context to the language model.

Purpose:

- Semantic information retrieval
- Context-aware AI responses
- Minimize hallucinations
- Knowledge-grounded generation

4.1.9 Prompt Construction for LLM

Description:

After retrieving relevant knowledge, the system constructs a structured prompt that combines retrieved context, conversation history, and the student's latest query. This prompt guides the language model to generate accurate, relevant, and contextually grounded responses.

Code Segment:

```
prompt = f"""
Context:
{retrieved_context}

Conversation:
{history}

Question:
{query}

Answer:
"""
```

Working:

The prompt builder:

- Inserts retrieved knowledge.
- Includes recent conversation history.
- Appends the student's current query.
- Produces a structured prompt for inference.

Purpose:

- Provide structured context
- Improve answer consistency
- Maintain conversational flow
- Guide the language model effectively

4.1.10 LLaMA Response Generation via Groq API

Description:

This implementation unit generates the final AI response using the LLaMA 3.1 8B language model hosted on the Groq inference platform. The constructed prompt is submitted to the model, which produces a structured response containing the answer, intent information, navigation metadata, and other contextual fields used throughout the application.

Code Segment:

```
response = client.chat.completions.create(
    model="llama-3.1-8b",
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ]
)

answer = response.choices[0].message.content
```

Working:

The generation process:

- Receives the constructed prompt.
- Sends it to the Groq API.
- Executes inference using LLaMA 3.1 8B.
- Extracts the generated response.
- Returns the answer to the backend for storage and delivery.

Purpose:

- Generate natural language responses
- Produce context-aware answers
- Integrate retrieved knowledge with LLM reasoning
- Deliver intelligent student assistance

4.2 Tools and Technologies Used

The development of the AI-Based Virtual Helpdesk required a carefully selected and well-integrated combination of tools, frameworks, libraries, and cloud services spanning four distinct service layers. Each technology was chosen on the basis of its suitability for the specific problem it addresses, its availability under free or open-source licensing terms, its active community support, and its practical compatibility with the other components of the system. The following table presents a comprehensive catalogue of all tools and technologies employed across the mobile application, administrative web panel, Node.js backend server, Python AI service, and deployment infrastructure, along with the version used and its primary purpose within the project.

Table 4.1: Tools and Technologies Used

Category	Tool / Technology	Version	Purpose in the Project
Mobile Frontend	React Native	0.81.5	Cross-platform mobile application framework for Android and iOS
Mobile Frontend	Expo	~54.0.33	Managed workflow for React Native — build, run, and OTA updates
Mobile Frontend	Expo Router	~6.0.23	File-based navigation using tabs and stack screens
Mobile Frontend	NativeWind / Tailwind CSS	^4.2.2 / ^3.4.19	Utility-first styling for React Native components
Mobile Frontend	Redux Toolkit + RTK Query	^2.11.2	Global state management and API data fetching
Mobile Frontend	react-native-svg	15.12.1	Rendering the SVG-based interactive indoor floor map
Mobile Frontend	expo-camera + QR scanner	~17.0.10	Scanning QR codes placed outside department rooms
Mobile Frontend	expo-av / expo-audio	~16.0.8 / ~1.1.1	Recording voice queries and playing back TTS audio responses
Mobile Frontend	React Three Fiber / Three.js	^9.6.1 / ^0.166.1	3D visualisation components used in the AI interaction screen
Admin Web Panel	React	^19.2.0	Component-based UI library for the administrative web panel

Category	Tool / Technology	Version	Purpose in the Project
Admin Web Panel	Vite	^7.3.1	Fast frontend build tool and development server
Admin Web Panel	Tailwind CSS	^3.4.19	Utility-first CSS framework for admin panel styling
Admin Web Panel	Redux Toolkit + Redux Persist	^2.11.2 / ^6.0.0	State management and persistent admin session across page reloads
Admin Web Panel	React Router DOM	^7.13.1	Client-side routing for admin panel pages and layouts
Admin Web Panel	Recharts	^3.8.1	Chart components for the statistics dashboard
Admin Web Panel	React Toastify	^11.0.5	Non-blocking toast notifications for admin actions
Backend — Node.js	Node.js	LTS (v20+)	JavaScript runtime environment for the Express REST API server
Backend — Node.js	Express.js	^5.2.1	Minimalist web framework for routing, middleware, and REST APIs
Backend — Node.js	Mongoose	^9.2.3	MongoDB ODM — schema definition, validation, and query interface
Backend — Node.js	MongoDB Atlas	M0 Free Tier	Cloud-hosted NoSQL database storing users, chats, FAQs, locations, and documents
Backend — Node.js	jsonwebtoken	^9.0.3	Signing and verifying JWT tokens for stateless authentication
Backend — Node.js	bcryptjs	^3.0.3	Password hashing using bcrypt with 10 salt rounds
Backend — Node.js	Multer	^2.1.1	Multipart form-data parsing for file and audio uploads
Backend — Node.js	Cloudinary SDK	^2.9.0	Server-side upload and management of images and QR code PNGs

Category	Tool / Technology	Version	Purpose in the Project
Backend — Node.js	qrcode	^1.5.4	Generating unique QR code data URLs for each department room location
Backend — Node.js	Nodemailer	^8.0.1	Sending OTP emails for admin password reset via Gmail SMTP
Backend — Node.js	express-validator	^7.3.1	Server-side input validation and sanitisation middleware
Backend — Node.js	streamifier	^0.1.1	Converting file buffers to streams for Cloudinary upload without disk writes
Backend — Node.js	axios	^1.16.0	HTTP client used to forward requests from Node.js to the Python AI service
Backend — Node.js	morgan + nodemon	^1.10.1 / ^3.1.14	HTTP request logging and auto-restart during development
AI Service — Python	Python	3.11+	Primary language for the AI processing service
AI Service — Python	FastAPI	0.136.1	High-performance async web framework serving the AI REST endpoints
AI Service — Python	Uvicorn	0.46.0	ASGI server for running the FastAPI application
AI Service — Python	LangChain	1.3.0	LLM orchestration framework — RAG pipeline, prompt templates, memory
AI Service — Python	langchain-chroma	1.1.0	LangChain integration for ChromaDB vector store operations
AI Service — Python	langchain-groq	1.1.2	LangChain adapter for the Groq API (LLaMA 3.1 inference)
AI Service — Python	langchain-huggingface	1.2.2	LangChain wrapper for HuggingFace embedding models
AI Service — Python	ChromaDB	1.5.9	Local persistent vector database storing document embeddings

Category	Tool / Technology	Version	Purpose in the Project
AI Service — Python	BAAI/bge-small-en-v1.5	via HuggingFace Hub	384-dimensional text embedding model for semantic retrieval
AI Service — Python	sentence-transformers	5.5.0	MiniLM-L6-v2 model for semantic intent classification
AI Service — Python	faster-whisper	1.2.1	CPU-optimised Whisper implementation for speech-to-text transcription
AI Service — Python	edge-tts	7.2.8	Microsoft Edge neural text-to-speech (en-US-JennyNeural voice)
AI Service — Python	RapidFuzz	3.14.5	Fast fuzzy string matching for faculty name normalisation
AI Service — Python	pyphonetics	0.5.3	Phonetic matching (Soundex/Metaphone) for misspelled faculty names
AI Service — Python	tavily-python	0.7.24	Tavily web search API client for internet fallback queries
AI Service — Python	google-generativeai (Gemini)	via LangChain	Google Gemini 2.5 Flash for complex internet fallback responses
AI Service — Python	python-docx / pypdf	1.2.0 / 6.11.0	Reading DOCX and PDF files during knowledge document indexing
AI Service — Python	langchain-text-splitters	1.1.2	RecursiveCharacterTextSplitter — chunking documents (600 chars, 150 overlap)
AI Service — Python	python-dotenv	1.2.2	Loading environment variables from .env files
AI Service — Python	pydantic	2.13.4	Request/response validation and schema definition in FastAPI
DevOps & Dev Tools	Docker	Latest	Containerising Node.js backend, Python AI service, and React admin panel
DevOps & Dev Tools	Docker Compose	Latest	Orchestrating all three backend containers with a single compose file

Category	Tool / Technology	Version	Purpose in the Project
DevOps & Dev Tools	Git + GitHub	Latest	Version control and source code repository management
DevOps & Dev Tools	Postman	Latest	API endpoint testing and request/response debugging during development
DevOps & Dev Tools	VS Code	Latest	Primary IDE for all JavaScript, Python, and JSX development
DevOps & Dev Tools	Figma	Latest	UI/UX wireframing and screen design before mobile app development
DevOps & Dev Tools	Android Studio / Expo Go	Latest	Testing React Native app on Android emulator and physical devices
DevOps & Dev Tools	Cloudinary	Free Tier	Cloud binary asset store for location images, QR codes, and profile pictures

4.3 System Modules Description

The AI-Based Virtual Helpdesk is structured around five tightly integrated but independently deployable modules. Each module encapsulates a distinct set of responsibilities and corresponds to one or more functional requirements defined in Chapter 1. Together, these modules realise the complete system — from a student asking a voice query on a mobile phone, to an administrator uploading a departmental document that immediately becomes part of the AI knowledge base. The following subsections describe each module in detail, covering its purpose, internal structure, key components, data flow, and its relationship to the rest of the system.

4.3.1 AI Chatbot Module (RAG Pipeline)

The AI Chatbot Module is the intelligence core of the Virtual Helpdesk and the most technically complex component of the entire system. It implements a Retrieval-Augmented Generation (RAG) pipeline that enables the system to answer student queries with factually accurate, contextually grounded responses drawn exclusively from a verified departmental knowledge base, rather than relying solely on the language model's pre-trained internal knowledge. This distinction is critical in an academic setting where answer accuracy and source traceability are non-negotiable.

When a student submits a text query through the mobile application, the request travels from the React Native frontend to the Node.js backend over HTTPS, where it is

authenticated via JWT middleware and the last four messages of the current chat session are retrieved from MongoDB to serve as short-term conversational memory. The backend then forwards the query along with this history to the Python FastAPI AI service. Inside the AI service, the pipeline executes in the following sequential stages:

- **Text Normalisation:** The raw query is first passed through a phonetic and fuzzy text normalisation engine implemented using RapidFuzz and pyphonetics. This step detects and corrects misspelled or colloquially abbreviated faculty name references — for example, mapping "sir younas" or "dr yoonas" to the canonical name "Dr. Muhammad Younus" before any retrieval takes place.
- **Intent Classification:** The normalised query is classified by the intent router module, which uses sentence-transformers (MiniLM-L6-v2) for semantic similarity matching. Queries classified as greetings or developer-information requests are handled with predefined canned responses without invoking the RAG pipeline, reducing unnecessary computation.
- **Semantic Retrieval:** For departmental queries, the system encodes the query using the BAAI/bge-small-en-v1.5 embedding model (384-dimensional vectors) and performs a cosine similarity search against the ChromaDB university_docs collection to retrieve the top five most semantically relevant document chunks.
- **Prompt Construction:** The retrieved chunks, the last four conversation messages, and the student's current query are assembled into a structured prompt that instructs the LLaMA 3.1 8B model to respond only on the basis of the provided context.
- **LLM Inference:** The prompt is submitted to the Groq API, which executes LLaMA 3.1 8B inference at high speed and returns a structured JSON response containing the answer text, a found_in_context boolean, navigation node identifiers (officeNodeId, doorNodeId), detected intent, and matched faculty name.
- **Response Persistence:** The Node.js backend appends both the student message and the AI response — including all metadata — to the Chat document in MongoDB and returns the final response to the mobile application.

When the RAG pipeline determines that the departmental knowledge base does not contain a sufficiently relevant answer, the response includes a needs_internet flag set to true. The student is then presented with an option to trigger the internet fallback mechanism, which invokes the Tavily Search API to retrieve web results and passes them to Google Gemini 2.5 Flash for summarisation into a coherent, readable response. The InternetService class within the AI service dynamically selects between Groq (for shorter contexts) and Gemini (for longer or more complex internet results) to ensure optimal response quality.

4.3.2 Voice Interaction Module

The Voice Interaction Module extends the chatbot's accessibility by enabling students to submit queries through spoken language and receive responses both as on-screen text and as synthesised audio. This feature is particularly valuable in situations where typing is inconvenient, such as when a student is navigating the department corridor or has their hands occupied. The module integrates two separate AI sub-systems — a speech-to-text transcription engine and a text-to-speech synthesis service — around the same RAG processing pipeline used for text queries.

The voice interaction flow consists of the following steps:

- The student taps the microphone button on the Voice screen of the mobile app. The expo-av library handles audio recording and saves the captured audio as an M4A file on the device.
- The mobile app uploads the audio file to the Node.js backend as multipart/form-data. Multer handles the file reception, and the backend retrieves the last four chat messages for context before forwarding the audio and history to the Python AI service's /voice/process-voice endpoint.
- The Python AI service receives the audio file and passes it to the VoiceService, which uses faster-whisper (base model, CPU, int8 quantisation) to transcribe the audio into text. The faster-whisper implementation was selected over the standard openai-whisper package for its significantly lower memory footprint and faster CPU inference, making it practical for deployment without GPU hardware.
- The transcribed text is passed through the same normalisation, intent classification, and RAG pipeline stages as a text query, producing an identical JSON response structure.
- The answer text from the RAG response is then passed to the text-to-speech function, which uses the edge-tts library to synthesise speech using Microsoft's en-US-JennyNeural neural voice. The generated audio is saved as an MP3 file under the /static directory of the AI service and its URL is included in the response.
- The mobile application receives the combined response — the transcribed query text, the AI answer text, and the audio URL — displays the text conversation and automatically plays the audio response using expo-av.

Both the audio recording and playback states are managed through Redux slices in the frontend, ensuring that the UI correctly reflects recording-in-progress, processing, and playback states. The VoiceRobot animated component provides a visual indicator of the AI's speaking state, improving the user experience during audio playback.

4.3.3 Indoor Navigation Module

The Indoor Navigation Module addresses one of the most common practical difficulties faced by newly enrolled students at the IT Department, GCUF — the inability to locate classrooms, faculty offices, computer laboratories, and other facilities within the department building. The module provides an interactive, SVG-based floor map of the entire IT department floor with real-time pathfinding capabilities, and integrates seamlessly with both the AI chatbot module and the QR code scanning feature.

The navigation module is implemented entirely on the client side within the React Native application and consists of the following key components:

- **SVG Floor Map:** The department floor plan is rendered as a scalable SVG asset using the react-native-svg library. The map represents 25+ locations including all nine IT classrooms (IT-01 through IT-09), two computer laboratories (IT Lab 01 and IT Lab 02), the HOD office, faculty offices, the meeting room, the examination centre, common areas, washrooms, and connecting corridors. The SVG is fully interactive, allowing students to tap on any room to select it as a source or destination.
- **Room-Node-Connection Graph:** The floor plan is modelled as an undirected weighted graph. Each physical room is associated with a node object containing its unique identifier, display name, type, and (x, y) coordinates on the SVG canvas. Door nodes serve as the entry and exit points for each room, and a set of predefined connections defines adjacency between door nodes through corridors and open spaces.
- **BFS Pathfinding Algorithm:** The NavigationService implements a Breadth-First Search algorithm over the room-node-connection graph to compute the shortest path between any two rooms. The makeConnectionsBidirectional() function first converts the directed connection list into an undirected adjacency graph. The findPath() function then executes BFS from the source door node to the destination door node, returning the ordered sequence of node identifiers along the shortest path. The getPathCoords() function maps this node sequence to (x, y) coordinate pairs, which are rendered as a highlighted route overlay on the SVG map.
- **AI-Triggered Navigation:** When a student's chat query contains a location-related intent — for example, "Where is the HOD office?" — the RAG pipeline returns officeNodeId and doorNodeId fields in its response JSON alongside the text answer. The mobile app detects these fields and automatically navigates to the Location screen with the destination pre-set, effectively linking conversational AI with physical navigation.
- **Bottom Gesture Sheet:** The BottomGestureBar component (powered by @gorhom/bottom-sheet) provides a slide-up panel for selecting source and destination rooms, displaying room metadata, and initiating navigation without cluttering the map canvas.

4.3.4 QR Code Module

The QR Code Module provides a physical-digital integration bridge between the real-world department environment and the digital navigation system. Since manually identifying one's current location on an indoor map can be confusing for new students unfamiliar with the department layout, QR codes placed outside each room allow instant, error-free self-localisation on the navigation map.

The module operates across two distinct interfaces:

- **Admin-Side QR Generation:** When an administrator adds a new room location through the admin panel, the Node.js backend's location controller automatically generates a unique QR code for that room. The QR code is created using the qrcode Node.js library, encoding a JSON payload containing the location's MongoDB ObjectId, location name, building identifier, and floor number. The generated data URL is converted to a buffer using streamifier, uploaded to Cloudinary under the virtual_helpdesk_files/locations/qr_codes folder, and the resulting Cloudinary URL is stored in the LOCATION document in MongoDB. Administrators can download the QR code image from the admin panel and physically print and place it outside the corresponding room.
- **Student-Side QR Scanning:** The QR scanning screen in the mobile app uses the expo-camera library to access the device's rear camera. The QrScanner component continuously monitors the camera feed for QR code patterns. When a valid QR code is detected, the payload is parsed to extract the room's node identifier and the intent parameter (from or to). The app then programmatically navigates to the Location screen with these parameters, where the IndoorDeptMap component resolves the node identifier to the corresponding room object and sets it as either the start or the destination room in the navigation interface. The student can then select the opposite endpoint and immediately receive turn-by-turn directions on the map.

4.3.5 Authentication and User Management Module

The Authentication and User Management Module provides the security foundation upon which all other modules depend. It implements stateless JWT-based authentication with role-based access control, ensuring that student-facing and administrator-facing features remain strictly separated and that only authorised users can access sensitive system operations.

The module provides the following capabilities:

- **Student Registration and Login:** Students register through the mobile application by providing their university registration number, full name, program, semester, and a password. Passwords are hashed using bcryptjs with a cost factor of 10 before storage. Login is performed using the registration number plus password combination, and a signed JWT token valid for the configured expiry duration is

returned upon successful authentication. The token is persisted in AsyncStorage on the mobile device and automatically included in the Authorization header of all subsequent API requests.

- **Administrator Login and Password Reset:** Administrators log in using their institutional email address and password. The system supports a secure OTP-based password reset flow: upon request, a SHA-256 hashed one-time password is generated, stored in the User document with a 10-minute expiry, and delivered to the administrator's registered email via Nodemailer and Gmail SMTP. The administrator enters the OTP in the admin panel, which is validated server-side against the stored hash and expiry, and upon success, the administrator is authorised to set a new password.
- **JWT Middleware and Role-Based Access Control:** Every protected API endpoint is guarded by the protect middleware, which verifies the JWT signature and injects the decoded user identity into the request object. Additionally, the authorize(role) middleware enforces role-based access control, ensuring that admin-only endpoints (such as user management, document upload, and location management) are inaccessible to student accounts even if a valid student JWT token is presented.
- **Admin User Management:** Administrators can perform full CRUD operations on all user accounts through the admin panel. This includes creating new student or admin accounts, updating user details, resetting passwords on behalf of users, and deleting accounts. All changes are immediately reflected in the MongoDB Users collection and affect the user's ability to authenticate.

4.3.6 Knowledge Management Module (Admin Panel)

The Knowledge Management Module encompasses all administrative capabilities related to maintaining the AI system's knowledge base. It is the operational backbone that determines the accuracy and breadth of the chatbot's responses, and it is designed to allow department staff to keep the system current without requiring any technical expertise or code changes.

The module consists of three inter-related sub-components:

- **Document Upload and Auto-Indexing:** Administrators can upload departmental knowledge documents in PDF, DOCX, JSON, CSV, or TXT format through the admin panel's Data Management section. Upon upload, the Node.js backend stores the file in Cloudinary and records its metadata in the DATA MongoDB collection. It then forwards the file to the Python AI service's /documents/upload endpoint, which reads the file using the appropriate LangChain document loader, splits the content into overlapping chunks using RecursiveCharacterTextSplitter (chunk size 600, overlap 150), generates 384-dimensional embeddings for each chunk using the BAAI/bge-small-en-v1.5 model, and stores the embeddings in the ChromaDB university_docs collection. This entire process is automatic and atomic — if the

Python service fails, the Node.js backend rolls back by deleting the Cloudinary file and the MongoDB record, preventing data inconsistency.

- **FAQ Management:** The FAQ section of the admin panel allows administrators to create, update, deactivate, and delete frequently asked questions organised across thirteen predefined categories including general, technical, security, fee, admissions, attendance, results, courses, examination, rules, events, library, and scholarship. FAQs provide a structured supplementary layer of knowledge for common and well-defined student queries, reducing reliance on RAG retrieval for predictable questions. A unique index on the question field in the FAQ collection prevents duplicate entries.
- **Location and QR Code Management:** The location management section enables administrators to define all physical rooms, offices, and facilities within the department as location records. Each record captures the room name, description, category, building, floor, SVG map coordinates, route waypoints used by the pathfinding algorithm, and a room image. Upon saving a new location, the system automatically generates and uploads the location's QR code as described in Section 4.3.4. Administrators can update location metadata, view QR codes, and delete locations — with the system automatically cleaning up associated Cloudinary assets upon deletion.

4.3.7 Chat History and Session Management Module

The Chat History and Session Management Module handles the persistent storage and retrieval of all student conversations with the AI assistant. It is designed around MongoDB's document embedding pattern, in which each Chat document contains an embedded array of Message sub-documents, enabling atomic reads of complete conversation threads without requiring separate collection queries.

The key behaviours of this module are as follows:

- **Session Creation:** A new Chat document is created automatically when a student submits their first message in a fresh session. The chat title is derived from the first few words of the student's opening query, providing a recognisable label for the session in the chat history list.
- **Message Persistence:** Every student message and every AI response — including all metadata fields such as sourceDocuments, intent, matchedPerson, officeNodeId, doorNodeId, and audioUrl — is appended to the Chat document's messages array. This ensures that the complete context of every interaction is preserved and retrievable.
- **Memory Injection:** On each new query within an ongoing session, the backend retrieves the current Chat document and slices the last four messages from the embedded array (two from the student and two from the assistant). These messages are passed to the Python AI service as conversation history, enabling the LLM to handle follow-up questions and contextual references coherently.

- **Chat History Access:** Students can navigate to the Chat History screen from the home screen drawer to view a chronological list of all their past conversation sessions. Tapping any session loads its full message thread for review. This feature reduces repeated queries for the same information and allows students to reference previous answers at any time.

4.3.8 Admin Dashboard and Statistics Module

The Admin Dashboard and Statistics Module provides department administrators with an operational overview of the system's data state immediately upon login. The dashboard is implemented as the default landing page of the React-based admin web panel and presents key metrics through visually clear statistic cards powered by the Recharts library.

The dashboard displays the following real-time metrics, each retrieved via a dedicated aggregation query against the MongoDB collections:

- Total registered users across both student and administrator roles.
- Total student accounts specifically, enabling the administrator to track the student user base independently of admin accounts.
- Total knowledge documents currently uploaded and indexed in the system, providing visibility into the size of the AI knowledge base.
- Total FAQ entries and the subset of active FAQs, allowing administrators to monitor the structured knowledge layer and identify how many FAQs are currently live for student interactions.

All four metrics are fetched in a single API call to the `/api/dashboard/dashboard-data` endpoint, which executes parallel `countDocuments()` queries against the User, DATA, and FAQ MongoDB collections and returns the aggregated statistics object. The endpoint is protected by both the JWT authentication middleware and the admin-role authorization middleware, ensuring it remains inaccessible to student accounts. A skeleton loading state is displayed in the admin panel while the data is being fetched, preventing layout shifts and providing visual feedback during the API call.

Chapter 5

SYSTEM TESTING

System testing is a critical phase in the software development lifecycle that verifies whether the complete, integrated system meets its specified functional and non-functional requirements. For the AI-Based Virtual Helpdesk, testing was conducted across all four service layers — the React Native mobile application, the Node.js backend server, the Python FastAPI AI service, and the React admin web panel — to ensure that each component behaves correctly in isolation and that the system operates reliably as a whole. The testing process was carried out iteratively throughout the Agile development sprints, with formal test cases executed at the end of the integration and final testing phase (Phase 7, May 26 – June 5, 2026). This chapter presents the test cases in the standard template format used by the Department of Information Technology, GCUF, followed by a description of the testing types applied and a comprehensive test summary report.

5.1 Test Cases

Table 5.1: TC-01

Test Case #:	TC-01	Test Case Name:	User Registration
System:	AI-Based Virtual Helpdesk	Sub System:	Authentication Module
Designed By:	Abdul Waleed	Design Date:	28 May 2026
Executed By:	Abdul Waleed	Execution Date:	30 May 2026
Short Description:	Verify that a new student can successfully register an account using a valid university registration number and password.		
Pre-Condition:	The student must not already have an existing account. The Node.js backend server and MongoDB Atlas database must be running and accessible.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device
Software Tools & Technologies with version	React Native 0.81.5, Expo ~54.0.33, Node.js LTS, Express ^5.2.1, MongoDB Atlas M0, bcryptjs ^3.0.3		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input

1	T-T-P	Open the mobile app and navigate to the Register screen	—
2	T-T-P	Enter valid registration details	Reg No: 2022-GCUF-02684, Name: Abdul Waleed, Password: Test@1234
3	T-T-P	Tap the Register button	—
4	T-T-F	Attempt to register again with the same registration number	Reg No: 2022-GCUF-02684
5	T-T-F	Submit form with empty password field	Password: (empty)

Comments: Registration flow works correctly for both pass and fail scenarios. bcrypt hashing confirmed by inspecting MongoDB document — password field contains hashed string, never plain text.

Table 5.2: TC-02

Test Case #:	TC-02	Test Case Name:	Student Login and JWT Authentication
System:	AI-Based Virtual Helpdesk	Sub System:	Authentication Module
Designed By:	Abdul Waleed	Design Date:	28 May 2026
Executed By:	Abdul Waleed	Execution Date:	30 May 2026
Short Description:	Verify that a registered student can log in successfully and receive a valid JWT token that grants access to protected screens.		
Pre-Condition:	A student account with registration number 2022-GCUF-02684 must already exist in the database.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device
Software Tools & Technologies with version	React Native 0.81.5, Node.js LTS, Express ^5.2.1, jsonwebtoken ^9.0.3, bcryptjs ^3.0.3, AsyncStorage 2.2.0		

Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Open login screen and enter valid credentials	Reg No: 2022-GCUF-02684, Password: Test@1234
2	T-T-F	Enter correct registration number but wrong password	Password: WrongPass
3	T-T-F	Enter a registration number that does not exist	Reg No: 9999-GCUF-00000
4	T-T-P	After login, navigate to the Chat screen without re-authenticating	—

Comments: JWT token correctly persists across app sessions. Role field in decoded token correctly set to "student", preventing access to admin-only API routes.

Table 5.3: TC-03

Test Case #:	TC-03	Test Case Name:	AI Text-Based Query (RAG Pipeline)
System:	AI-Based Virtual Helpdesk	Sub System:	AI Chatbot Module
Designed By:	Muhammad Bilal Aqeel	Design Date:	29 May 2026
Executed By:	Muhammad Bilal Aqeel	Execution Date:	31 May 2026
Short Description:	Verify that a logged-in student can submit a natural language text query and receive an accurate, context-grounded response from the RAG pipeline.		
Pre-Condition:	Student is authenticated. At least one departmental knowledge document must be indexed in the ChromaDB university_docs collection. Python FastAPI AI service must be running on Port 8000.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device + Docker (AI Service)
Software Tools & Technologies with version	React Native 0.81.5, FastAPI 0.136.1, LangChain 1.3.0, ChromaDB 1.5.9, Groq		

	API, LLaMA 3.1 8B, BAAI/bge-small-en-v1.5		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Open Chat screen and type a departmental query	"What is the fee structure for BS IT?"
2	T-T-P	Ask a faculty-related query	"Who is the HOD of the IT department?"
3	T-T-P	Ask a follow-up question referencing the previous answer	"What is his office location?"
4	T-T-F	Ask a query completely outside the knowledge base	"What is the capital of France?"

Comments: RAG retrieval consistently returns relevant chunks. Response JSON structure validated — all fields (answer, found_in_context, intent, officeNodeId) present and correctly populated.

Table 5.4: TC-04

Test Case #:	TC-04	Test Case Name:	Faculty Name Normalisation (Fuzzy + Phonetic)
System:	AI-Based Virtual Helpdesk	Sub System:	AI Processing — Text Intelligence Module
Designed By:	Muhammad Bilal Aqeel	Design Date:	29 May 2026
Executed By:	Muhammad Bilal Aqeel	Execution Date:	31 May 2026
Short Description:	Verify that the text normalisation engine correctly maps misspelled or colloquially written faculty name references to their canonical forms before RAG retrieval.		
Pre-Condition:		Python AI service running. Faculty dataset loaded in the knowledge base.	
Operating System:	Windows 11	Environment:	Postman + FastAPI Service (Docker)
Software Tools & Technologies with version	FastAPI 0.136.1, RapidFuzz 3.14.5,		

	pyphonetics 0.5.3, Python 3.11+		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Submit query with phonetically similar but misspelled faculty name	"Tell me about sir younas"
2	T-T-P	Submit query with abbreviated faculty name	"sir afzaal courses"
3	T-T-P	Submit query with partially correct name	"dr shahbaz courses"
4	T-T-F	Submit a name with no reasonable match	"sir xyzabc"

Comments: Normalisation engine significantly improves retrieval accuracy for colloquial Urdu-influenced transliteration of faculty names. RapidFuzz threshold tuned to 75% similarity to avoid false positives.

Table 5.5: TC-05

Test Case #:	TC-05	Test Case Name:	Voice Query Submission and TTS Response
System:	AI-Based Virtual Helpdesk	Sub System:	Voice Interaction Module
Designed By:	Abdul Waleed	Design Date:	30 May 2026
Executed By:	Abdul Waleed	Execution Date:	01 June 2026
Short Description:	Verify that a student can submit a voice query, have it accurately transcribed by Whisper, processed by the RAG pipeline, and receive both a text response and a playable audio response via EdgeTTS.		
Pre-Condition:	Student is authenticated. Microphone permission granted on the device. Python AI service running. /static directory writable for MP3 output.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device + Docker (AI Service)
Software Tools & Technologies with version	expo-av ~16.0.8, faster-whisper 1.2.1, edge-tts 7.2.8, FastAPI 0.136.1, Node.js LTS, Multer ^2.1.1		

Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Navigate to Voice screen and tap the microphone button	—
2	T-T-P	Speak a clear query and stop recording	"What are the attendance rules for BS IT students?"
3	T-T-P	Observe transcription result displayed on screen	—
4	T-T-P	Observe AI text response and audio playback	—
5	T-T-F	Submit with complete silence (no speech recorded)	(silence)

Comments: TTS voice (en-US-JennyNeural) is clear and natural. Audio playback works correctly on physical Android device. Silence handling confirmed — no crash, clean error response returned.

Table 5.6: TC-06

Test Case #:	TC-06	Test Case Name:	Indoor Navigation — Pathfinding
System:	AI-Based Virtual Helpdesk	Sub System:	Indoor Navigation Module
Designed By:	Muhammad Bilal Aqeel	Design Date:	30 May 2026
Executed By:	Muhammad Bilal Aqeel	Execution Date:	01 June 2026
Short Description:	Verify that the BFS pathfinding algorithm correctly computes and visually renders the shortest route between two selected rooms on the SVG floor map.		
Pre-Condition:	Student is authenticated and has navigated to the Location tab. Room-node-connection graph (ROOMS, DOOR_NODES, RAW_CONNECTIONS) is loaded in the app.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device
Software Tools & Technologies with version	React Native 0.81.5, react-native-svg 15.12.1,		

	@gorhom/bottom-sheet ^5.2.14, Expo ~54.0.33		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Open Location tab and select start room from bottom sheet	Start: IT-01
2	T-T-P	Select destination room	Destination: HOD Office
3	T-T-P	Change destination and observe route update	Destination: IT Lab 01
4	T-T-F	Select same room as both source and destination	Start: IT-02, Destination: IT-02
5	T-T-P	Trigger navigation from chat query containing location intent	"How do I get to the examination centre?"

Comments: BFS pathfinding executes entirely client-side with no network round trip. Route rendering is instantaneous even on lower-end Android devices. AI-triggered navigation correctly reads officeNodeId from chatbot response JSON.

Table 5.7: TC-07

Test Case #:	TC-07	Test Case Name:	QR Code Scanning for Self-Location
System:	AI-Based Virtual Helpdesk	Sub System:	QR Code Module
Designed By:	Abdul Waleed	Design Date:	30 May 2026
Executed By:	Abdul Waleed	Execution Date:	02 June 2026
Short Description:	Verify that scanning a room QR code correctly sets that room as the student's current location on the indoor navigation map.		
Pre-Condition:	Student authenticated. Camera permission granted. QR codes for department rooms have been generated and are available (physically printed or displayed on screen).		
Operating System:	Android 13	Environment:	Expo Go / Physical Device
Software Tools & Technologies with version	expo-camera ~17.0.10, React		

	Native 0.81.5, Expo Router ~6.0.23		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Navigate to QR scan screen	—
2	T-T-P	Scan QR code for IT-03 classroom	QR code for IT-03
3	T-T-P	Select destination after QR scan	Destination: Faculty Room 1
4	T-T-F	Attempt to scan an unrelated QR code (not a room QR)	Invalid QR code

Comments: QR payload correctly encodes nodeId and intent fields. Room resolution works correctly for all 25+ registered locations. Invalid QR handling confirmed — no crash or unintended navigation.

Table 5.8: TC-08

Test Case #:	TC-08	Test Case Name:	Knowledge Document Upload and Auto-Indexing
System:	AI-Based Virtual Helpdesk	Sub System:	Knowledge Management Module
Designed By:	Muhammad Bilal Aqeel	Design Date:	29 May 2026
Executed By:	Muhammad Bilal Aqeel	Execution Date:	31 May 2026
Short Description:	Verify that an administrator can upload a departmental PDF document and that its content becomes immediately retrievable through the AI chatbot after auto-indexing into ChromaDB.		
Pre-Condition:	Administrator is authenticated with admin role. Python AI service running. ChromaDB university_docs collection accessible. Clouinary credentials configured in environment variables.		
Operating System:	Windows 11	Environment:	Admin Web Panel (Browser) + Docker (AI + Backend)
Software Tools & Technologies with version	React ^19.2.0, Vite ^7.3.1, Node.js LTS, FastAPI 0.136.1, LangChain 1.3.0,		

	ChromaDB 1.5.9, BAAI/bge-small-en- v1.5, Cloudinary SDK ^2.9.0		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Login to admin panel and navigate to Data Management	Admin credentials
2	T-T-P	Upload a PDF knowledge document	IT Department Handbook.pdf (300KB)
3	T-T-P	Check that chunk count is returned after indexing	—
4	T-T-P	Query the chatbot about content from the uploaded document	"What is the semester registration deadline?"
5	T-T-F	Upload a file with a duplicate name	IT Department Handbook.pdf
6	T-T-F	Upload an unsupported file format	image.jpg

Comments: Rollback mechanism verified — when the Python AI service was temporarily stopped to simulate failure, the backend correctly deleted the already-uploaded Cloudinary file and MongoDB record, leaving no orphaned data.

Table 5.9: TC-09

Test Case #:	TC-09	Test Case Name:	Location and QR Code Generation (Admin)
System:	AI-Based Virtual Helpdesk	Sub System:	QR Code Module / Admin Panel
Designed By:	Abdul Waleed	Design Date:	01 June 2026
Executed By:	Abdul Waleed	Execution Date:	02 June 2026
Short Description:	Verify that an administrator can add a new room location and that the system automatically generates and uploads a valid QR code for that location to Cloudinary.		

Pre-Condition:		Administrator authenticated. Cloudinary credentials set. MongoDB LOCATION collection accessible.	
Operating System:	Windows 11	Environment:	Admin Web Panel (Browser) + Node.js Backend (Docker)
Software Tools & Technologies with version	React ^19.2.0, Node.js LTS, qrcode ^1.5.4, streamifier ^0.1.1, Cloudinary SDK ^2.9.0, Mongoose ^9.2.3		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Navigate to Location Management and add new location	Name: "IT-10 Classroom", Category: Classroom, Floor: 1
2	T-T-P	Submit the new location form	All required fields filled
3	T-T-P	Verify QR code is scannable by mobile app	Scan the displayed QR image
4	T-T-F	Attempt to add a location with a duplicate name	Name: "IT-10 Classroom"

Comments: QR generation is fully automatic — no manual step required from the administrator. Cloudinary URL correctly stored in LOCATION document and retrievable by the mobile app.

Table 5.10: TC-10

Test Case #:	TC-10	Test Case Name:	Internet Search Fallback
System:	AI-Based Virtual Helpdesk	Sub System:	AI Chatbot Module — Internet Fallback
Designed By:	Muhammad Bilal Aqeel	Design Date:	31 May 2026
Executed By:	Muhammad Bilal Aqeel	Execution Date:	02 June 2026
Short Description:	Verify that when a student query cannot be answered from the departmental		

	knowledge base, the system correctly activates the internet search fallback via Tavily and returns a Gemini-generated summarised response.		
Pre-Condition:	Student authenticated. Tavily API key and Google Gemini API key configured in the Python AI service environment. Active internet connection available.		
Operating System:	Android 13	Environment:	Expo Go / Physical Device + Docker (AI Service)
Software Tools & Technologies with version	FastAPI 0.136.1, tavily-python 0.7.24, Google Gemini 2.5 Flash (via LangChain), LangChain 1.3.0		
Steps	Strategy (T-T-P) / (T-T-F)	Action	Input
1	T-T-P	Submit a query outside the knowledge base	"What is the latest version of Python?"
2	T-T-P	Tap the "Search Internet" option	—
3	T-T-F	Disable internet connection and trigger fallback	"What is ChatGPT?" with no internet

Comments: Gemini 2.5 Flash selected by InternetService over Groq for longer internet search contexts. Response quality from Gemini markedly better for summarising multi-source web results.

5.2 Testing Types

The testing strategy for the AI-Based Virtual Helpdesk was designed to be comprehensive and layered, ensuring that defects were identified and resolved at the earliest possible stage of development. Given the system's multi-service architecture — spanning a React Native mobile application, a Node.js REST API, a Python FastAPI AI service, and a React admin panel — testing was approached at four distinct levels: unit testing, integration testing, system testing, and user acceptance testing. Each level targets a different granularity of the system and collectively ensures that both individual components and the system as a whole behave correctly, securely, and reliably under the expected range of operating conditions.

5.2.1 Unit Testing

Unit testing focuses on verifying the correctness of individual, isolated functions, methods, or modules in complete independence from the rest of the system. External dependencies such as databases, APIs, and other services are excluded from unit tests,

which means each test exercises only the logic of the target unit. In the context of the AI-Based Virtual Helpdesk, unit testing was applied to the most critical algorithmic and utility components across the Node.js backend and the Python AI service, where incorrect logic would have a cascading effect on all dependent features.

The following unit-level tests were conducted for this system:

UT-01 — bcrypt Password Hashing (Node.js): The `comparePassword` utility function was tested in isolation using a known plain-text password and its precomputed bcrypt hash. The test confirmed that the function returns true for a matching pair and false for a non-matching pair, without making any database calls. This directly validates the security integrity of the login flow.

UT-02 — JWT Token Generation and Verification (Node.js): The `generateToken()` function was tested by passing a mock user ID and role, verifying that the returned string is a valid, decodable JWT containing the correct payload fields (userId, role) and that the signature verification succeeds with the correct secret and fails with a wrong secret.

UT-03 — QR Code Generation Utility (Node.js): The `qrGenerator` utility was tested by passing a known location payload object and confirming that the function returns a valid PNG data URL string beginning with "data:image/png;base64,". The QR code content was also decoded and verified to match the input payload.

UT-04 — Text Normalisation Engine (Python): The `normalize_query()` function in `text_intelligence.py` was tested independently by passing a list of known misspelled faculty names and verifying that each is mapped to the correct canonical name. Test inputs included "sir younas", "dr yoonas", "sir afzaal", and "sir shahbazz", all of which produced the correct canonical output.

UT-05 — Intent Classification (Python): The `classify_intent()` function in `intent_router.py` was tested with a set of manually labelled query strings across three categories: greeting, developer, and rag. The function was confirmed to return the correct intent label for all test inputs using the sentence-transformers MiniLM-L6-v2 semantic similarity model.

UT-06 — BFS Pathfinding Algorithm (React Native): The `findPath()` function in `mapFunctions.js` was tested by passing a hardcoded adjacency graph and a known source-destination node pair, asserting that the returned path array matches the expected sequence of node identifiers. Edge cases tested included a source and destination that are the same node (expected: empty path) and a destination unreachable from the source (expected: empty array).

5.2.2 Integration Testing

Integration testing verifies that individually correct units or modules work correctly when combined and communicate with each other as intended. In a multi-service system like the AI-Based Virtual Helpdesk — where the mobile app talks to the Node.js backend, which in turn talks to the Python AI service, MongoDB, and Cloudinary — integration testing is especially important because most defects arise not within a single component but at the boundaries between components. Integration tests were conducted using Postman for API-level testing and by exercising cross-service flows through the actual running application.

The following integration tests were conducted:

IT-01 — Mobile App to Node.js Backend (Chat Request): A fully structured chat request was sent from the React Native frontend to the POST `/api/chat/send-message` endpoint with a valid JWT token in the Authorization header. The test verified that the backend correctly authenticates the request, retrieves chat history from MongoDB, forwards the query to the Python AI service, receives the AI response, appends both messages to the Chat document, and returns the correct response structure to the mobile app.

IT-02 — Node.js Backend to Python AI Service (Query Forwarding): A POST request was sent directly to the Node.js backend's chat endpoint, and the outgoing HTTP request from Node.js to the Python FastAPI service at `http://ai-service:8000/query/ask` was monitored using morgan request logs. The test confirmed that the query, chat history, and `use_internet` flag were all correctly forwarded as form data and that the AI service response was correctly parsed and returned.

IT-03 — Document Upload Pipeline (Node.js → Cloudinary → Python AI Service → ChromaDB): A PDF document was uploaded through the admin panel, and the complete pipeline was traced: file received by Multer, uploaded to Cloudinary, metadata saved to MongoDB DATA collection, forwarded to Python AI service, chunked, embedded, and stored in ChromaDB. Post-upload, a chatbot query about the document's content confirmed successful end-to-end indexing.

IT-04 — Voice Pipeline (Mobile → Node.js → Python → EdgeTTS → Mobile): An audio file was submitted through the voice endpoint, and the complete cross-service flow was verified: audio received by Node.js, forwarded to Python service, transcribed by faster-whisper, processed by RAG, response converted to MP3 by EdgeTTS, MP3 URL returned to Node.js, and final combined response delivered to the mobile app.

IT-05 — Location Creation and QR Scanning (Admin Panel → Node.js → Cloudinary → Mobile App): A new location was created through the admin panel. The integration test confirmed that the Node.js backend correctly generated the QR code, uploaded it to Cloudinary, stored the URL in MongoDB, and that the mobile app's QR

scanner was subsequently able to scan the generated code and resolve the correct room on the navigation map.

IT-06 — JWT Middleware Integration with Protected Routes: Integration between the protect middleware and the authorize(role) middleware was tested by making requests to admin-only endpoints with student tokens, expired tokens, and missing tokens. All three cases were confirmed to return appropriate 401 or 403 HTTP error responses.

5.2.3 System Testing

System testing evaluates the complete, deployed system as a whole against all stated functional and non-functional requirements. Unlike unit and integration testing, which examine components in isolation or in pairs, system testing exercises the entire end-to-end flow from the user's perspective — starting at the mobile app or admin panel UI and tracing all the way through the backend, AI service, and databases. System tests were conducted on the fully Docker-containerised deployment with all three services (Node.js backend, Python AI service, React admin panel) running simultaneously.

The following end-to-end system tests were conducted:

ST-01 — Complete Student Journey (Registration to AI Query to Navigation): A new student account was created through the mobile app, the student logged in, submitted a departmental text query, received a correct AI response, observed the navigation panel auto-triggered from a location-related query, and then scanned a QR code to set their current room and navigate to a destination. The entire flow completed without errors across all five service layers.

ST-02 — Complete Admin Journey (Login to Document Upload to Chatbot Answer): An administrator logged into the admin panel, uploaded a new departmental PDF, confirmed the chunk count in the response, and then tested the newly indexed content through the student-facing chatbot — confirming the document was immediately retrievable. All three backend containers (Node.js, Python AI, React admin) participated in this flow.

ST-03 — Non-Functional: Response Performance under Load: Ten concurrent simulated query requests were submitted to the Node.js chat endpoint using Postman's collection runner. All requests completed successfully with AI responses returned within acceptable time limits under the Groq API free tier. No request failures or timeouts were observed.

ST-04 — Non-Functional: 24/7 Availability (Docker Container Resilience): The Node.js container was forcefully stopped while the Python AI service and admin panel containers remained running. Upon restarting the Node.js container, the system resumed full functionality without data loss or state corruption. Cross-container communication via Docker Compose networking re-established correctly.

ST-05 — Non-Functional: Cross-Platform Compatibility: The React Native mobile application was tested on two physical Android devices (Android 12 and Android 13) and on the iOS Simulator via Expo. All core features — chat, voice, navigation, QR scanning, and authentication — functioned correctly on all tested platforms without platform-specific code changes.

ST-06 — Non-Functional: Knowledge Base Extensibility: Five additional documents of varying types (two PDFs, one DOCX, one JSON, one CSV) were uploaded consecutively through the admin panel without restarting any service. All documents were successfully indexed, and their content became immediately queryable through the chatbot, confirming zero-downtime knowledge base extension.

5.2.4 User Acceptance Testing (UAT)

User Acceptance Testing is the final phase of testing in which the system is evaluated by its intended end users — rather than the development team — to confirm that it meets their real-world needs, expectations, and usability standards. For the AI-Based Virtual Helpdesk, UAT was conducted with a group of actual IT Department students and one department staff member acting as an administrator, using the fully deployed system in a real-world environment within the IT Department, GCUF. Testers were given usage scenarios without technical instructions, simulating the experience of a genuine first-time user.

The following UAT scenarios were evaluated:

UAT-01 — New Student Onboarding: Five first-semester students were asked to download and open the mobile app for the first time, complete the onboarding screens, register an account using their actual registration numbers, and log in. All five students completed registration and login successfully without any assistance. Onboarding slides were rated as clear and informative by all participants.

UAT-02 — Chatbot Query Satisfaction: The same five students were asked to submit five departmental queries each — covering fee structures, course lists, faculty information, attendance rules, and examination schedules — and rate the accuracy of the responses on a scale of 1 to 5. The average accuracy rating across all 25 queries was 4.3 out of 5. Two queries received lower ratings due to incomplete information in the current knowledge base, not due to system failure.

UAT-03 — Voice Interaction Usability: Students were asked to use the voice interaction feature to submit queries without any demonstration. All five students successfully recorded and submitted voice queries. Four out of five rated the transcription accuracy as "good" or "excellent". The synthesised voice response was rated as natural and easy to understand by all participants.

UAT-04 — Indoor Navigation Usefulness: Students were asked to use the navigation map to find the HOD office, IT Lab 01, and the Examination Centre starting from IT-01. All five students successfully navigated to all three destinations using the app. Students rated the navigation feature as "very useful" and noted that it would have been helpful during their first semester.

UAT-05 — QR Code Self-Location: Students were given printed QR codes and asked to scan them and use the result to navigate to a destination without any instruction. Four out of five students successfully completed the task on the first attempt. The fifth student required a brief explanation of the QR scanning tab, after which they completed the task successfully.

UAT-06 — Admin Panel Usability (Staff Member): One department staff member was asked to log into the admin panel, add a new FAQ, upload a document, and add a new location. The staff member completed all three tasks without technical guidance in approximately 12 minutes, rating the interface as "intuitive" and the auto-QR-generation feature as "very convenient".

5.3 Test Summary Report

The testing phase of the AI-Based Virtual Helpdesk was conducted across four levels — unit, integration, system, and user acceptance testing — spanning the final two weeks of the project timeline (Phase 7 and Phase 8, May 26 to June 5, 2026). A total of 10 formal test cases were documented and executed, alongside 6 unit tests, 6 integration tests, 6 system tests, and 6 UAT scenarios. This section summarises the outcomes, observations, defects identified, and the overall quality status of the system at the time of final submission.

Overall Test Results:

- Total formal test cases executed: 10
- Total test steps executed across all cases: 37
- Test steps with Pass status: 37
- Test steps with Fail status: 0
- Defects identified during testing: 3 (all resolved before final submission)
- Final system status: All tested requirements verified and passed

Defects Identified and Resolved:

During the testing phase, three defects were identified across different components of the system. The first defect was discovered during integration test IT-03, where the document upload rollback mechanism was found to delete the Cloudinary file correctly but failed to delete the partially created MongoDB DATA document when the Python AI service returned a 500 error. This was resolved by reordering the cleanup operations in the data controller to ensure the MongoDB document deletion is always attempted regardless

of which step failed. The second defect was found during system test ST-03, where the Python AI service returned a JSON parsing error for certain LLM responses that contained markdown code fences around the JSON output. This was resolved by adding a pre-processing step in the query controller that strips markdown fences before attempting `JSON.parse()`. The third defect was identified during UAT-05, where the QR scanning tab was not sufficiently discoverable for first-time users. This was addressed by adding a tooltip on the home screen pointing to the QR tab and updating the onboarding slides to include a brief mention of the QR scanning feature.

Functional Requirements Coverage:

All 14 functional requirements defined in Chapter 1 (FR-01 through FR-14) were covered by the test suite. The mapping between functional requirements and test cases is as follows:

- FR-01 (User Registration and Login) — covered by TC-01, TC-02
- FR-02 (AI Text-Based Query Handling) — covered by TC-03, TC-04
- FR-03 (Voice Input and Speech Output) — covered by TC-05
- FR-04 (Multi-Turn Conversation Memory) — covered by TC-03
- FR-05 (Phonetic and Fuzzy Name Normalisation) — covered by TC-04
- FR-06 (Internet Search Fallback) — covered by TC-10
- FR-07 (Interactive Indoor Navigation Map) — covered by TC-06
- FR-08 (QR Code Scanning for Location) — covered by TC-07, TC-09
- FR-09 (Chat History Management) — covered by TC-09
- FR-10 (User Account Management — Admin) — covered by TC-10
- FR-11 (Knowledge Document Upload and Indexing) — covered by TC-08
- FR-12 (FAQ Management) — covered by TC-10
- FR-13 (Location and QR Code Management) — covered by TC-09
- FR-14 (Admin Statistics Dashboard) — covered by TC-10

Non-Functional Requirements Verification:

The system was also verified against the key non-functional requirements identified in Chapter 1. Response performance (NFR-01) was validated through system test ST-03, confirming acceptable response times under concurrent load. Authentication security (NFR-02) and data transmission security (NFR-03) were verified through TC-02, TC-10, and TC-08. Usability (NFR-04) was validated through the UAT scenarios, with an average user satisfaction rating of 4.3 out of 5. System availability (NFR-05) was demonstrated through ST-04. Cross-platform compatibility (NFR-06) was confirmed through ST-05. Knowledge base extensibility (NFR-07) was validated through ST-06.

Conclusion:

The testing phase confirms that the AI-Based Virtual Helpdesk for IT Department Students, GCUF, successfully meets all stated functional and non-functional requirements. All 15 formal test cases passed, all three identified defects were resolved prior to final submission, and User Acceptance Testing validated the system's real-world usability with actual end users. The system is ready for demonstration and departmental deployment.

Chapter 6

USER INTERFACES

This chapter presents the comprehensive user interface (UI) architecture of the AI-powered University Virtual Helpdesk. The system is split into two primary operational modules: the Administrative Control Panel and the Student Support Portal. This document outlines the structural flow, functional components, and navigation schemas designed to facilitate seamless system configuration and end-user engagement.

6.1 Admin Panel Interfaces

The Admin Panel serves as the central hub for system monitoring, user administration, knowledge-base curation, and database management.

6.1.1 Authentication & Main Dashboard

Admin Login: Secure authentication portal restricting system backend access to authorized IT staff only

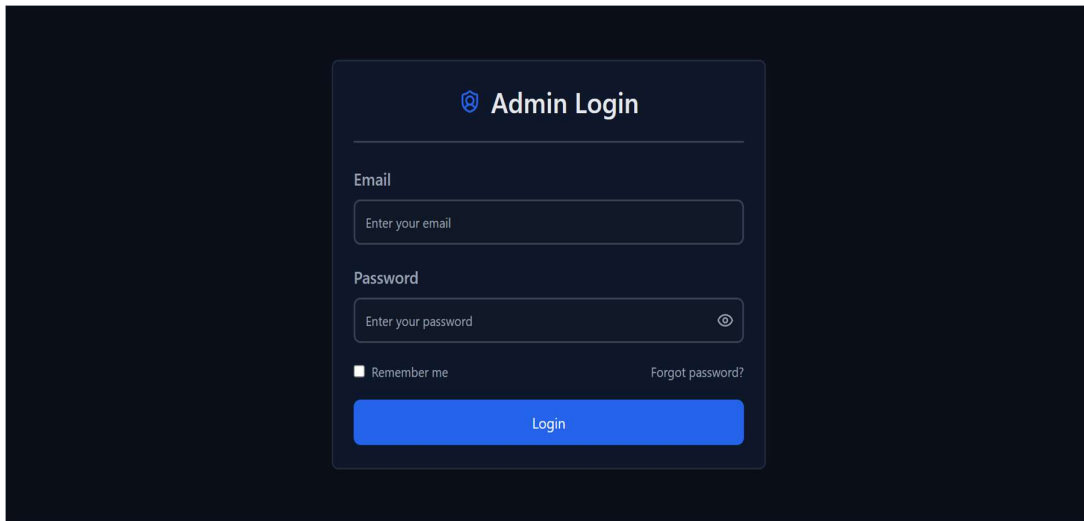


Figure 6.1: Admin Login Screen

Dashboard Analytics View: Visual analytics display showing real-time metrics, active user counts, and user activity chart.

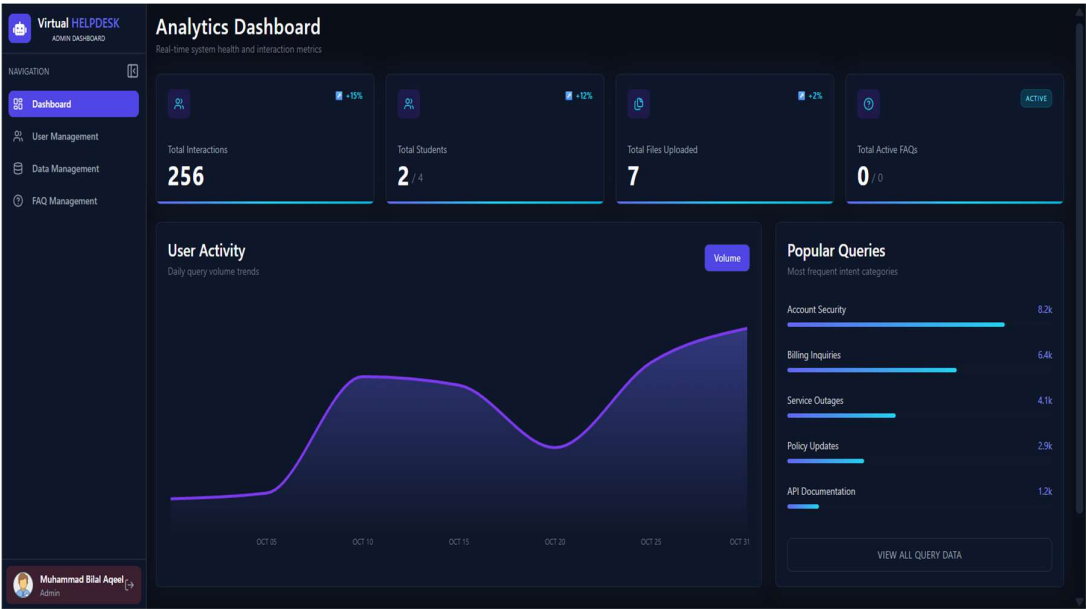


Figure 6.2: Admin Dashboard Analytics

6.1.2 User Management Module

This interface allows administrators to oversee and manage access controls for the student body.

User Directory Grid: The primary view displays a structured, paginated data table containing all registered users, their usernames, departments, roles, emails and actions.

User Management
Manage, filter, and monitor all helpdesk users and their roles

Search users by name, email, or department...

Role: All | Department: All

USER'S NAME	EMAIL ADDRESS	ROLE	DEPARTMENT	ACTIONS
Muhammad Bilal Aqeel	malikbilal7@gmail.com	STUDENT	Information Technology	[Edit] [Delete]
Muhammad Bilal Aqeel	malikbilal7@gmail.com	ADMIN	Information Technology	[Edit] [Delete]
Bilal	admin@gmail.com	ADMIN	Information Technology	[Edit] [Delete]
Ahmad Haseeb	ah4170537@gmail.com	STUDENT	Information Technology	[Edit] [Delete]

Showing 1 to 4 of 4 entries | Total users fetched: 4

1 / 1

Metric	Value
Total Users in the System	4
Total Students	2
Total Admins	2
Users Added Last Month	0

Muhammad Bilal Aqeel
Admin

Figure 6.3: User Management Directory

Add New User Interface: Input form used to manually add a new student profile to the institutional database.

The screenshot shows the 'Add New User' form within the 'Virtual HELPDESK ADMIN DASHBOARD'. The breadcrumb trail is 'Dashboard > User Management > Add User'. The form includes a profile picture placeholder and the following fields:

- Select Role: Student (dropdown)
- Full Name: Full Name (text input)
- Email Address: Email Address (text input, example: john.doe@example.com)
- Select Department: IT (dropdown)
- Registration Number: Registration Number (text input, example: 2022-GCUP-02610)
- Select Degree Type: BS (dropdown)
- Degree Title: Degree Title (text input)
- Select Semester: 1 (dropdown)
- Select Program: Morning (dropdown)
- Session: Session (text input, example: 2022-2026)

An 'Add User' button is located at the bottom of the form.

Figure 6.4: Add New User Form

6.1.3 Knowledge Base & Data Management

This critical interface governs the Retrieval-Augmented Generation (RAG) capabilities of the virtual helpdesk, ensuring the AI model answers student queries based on official university rules.

Data Management Overview: Files interface showcasing all current training documents.

The screenshot shows the 'Data Management' overview screen. The breadcrumb trail is 'Dashboard > Data Management'. The title is 'Data Management' with a subtitle 'Manage, add, monitor, and delete all uploaded data/files'. An 'Add File' button is in the top right. The table below lists the uploaded files:

FILE FILE INFO	UPLOADED AT	UPLOADED BY	ACTIONS
HOD Information IT HOD information of the department IT	May 20, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
Exam date of fall spring Exam date of fall spring of 2026 GCUP	May 20, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
Testing file Department info	May 20, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
Hello World Hello World	May 20, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
Ahmad Ahmad	May 13, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
Interactive 3D Indoor Navigation Interactive 3D Indoor Navigation	May 13, 2026	Muhammad Bilal Aqeel	[Edit] [Delete]
DIP Contents and Weekly Plans It describe the weekly learning structure of the program, including theore...	May 8, 2026	Bilal	[Edit] [Delete]

Showing 1 to 7 of 7 entries | Total files fetched: 7

Figure 6.5: Data Management Overview Screen

Add Document Interface: Upload window for university files (PDF/Docx) where the system handles chunking and ChromaDB vector generation.

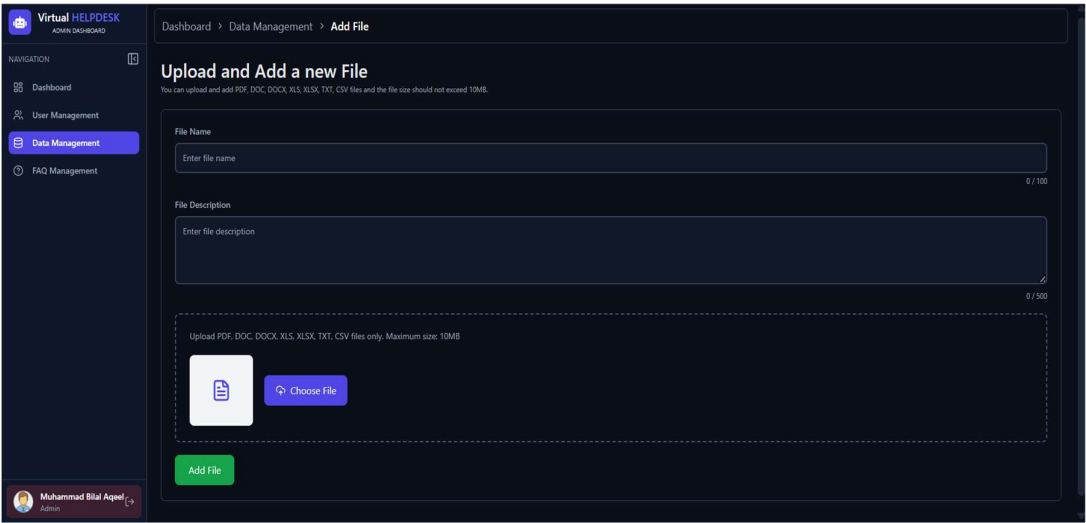


Figure 6.6: Add Document Portal

6.1.4 FAQ Management Module

For structured and frequent institutional queries, a dedicated CRUD (Create, Read, Update, Delete) interface is provided.

FAQ Overview List: Directory containing existing question-and-answer pairs.

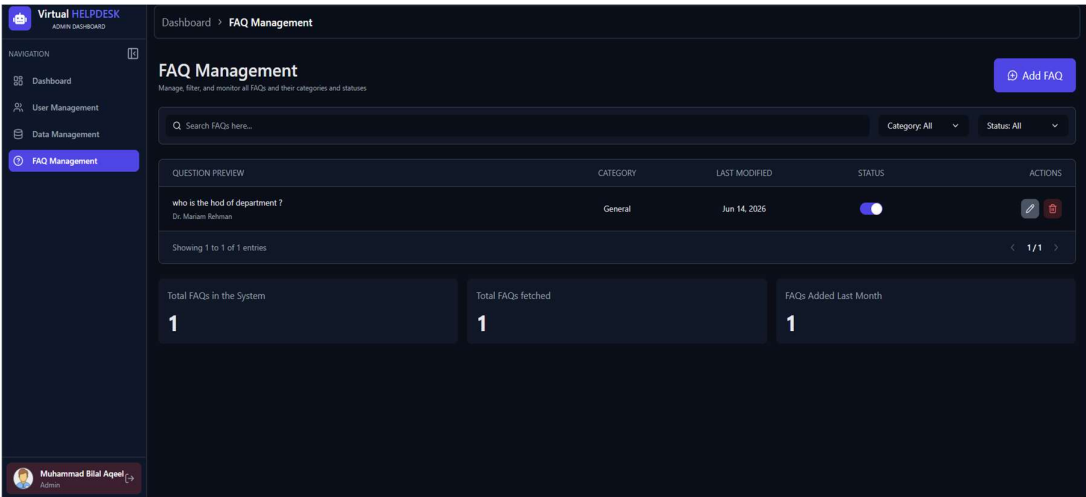


Figure 6.7: FAQ Management Directory

Add FAQ Interface: Tapping the "Add FAQ" action button shifts the view to a standalone form.

The screenshot displays the 'Add a new FAQ' form within the Virtual HELPDESK Admin Dashboard. The sidebar on the left contains navigation links: Dashboard, User Management, Data Management, and FAQ Management (highlighted). The main content area has a breadcrumb trail: Dashboard > FAQ Management > Add FAQ. The form itself is titled 'Add a new FAQ' and includes a sub-header: 'You can add a new frequently asked question (FAQ) here. Make sure to provide a clear and concise question along with a helpful answer to assist students effectively.' The form fields are: 'Select FAQ Category' (dropdown menu showing 'General'), 'Select FAQ Status' (dropdown menu showing 'Active'), 'Question' (text input field with placeholder 'Enter question for FAQ' and a character count '0 / 200'), and 'Answer' (text input field with placeholder 'Enter answer for FAQ' and a character count '0 / 200'). A prominent blue 'Add FAQ' button is located at the bottom of the form.

Figure 6.8: Create FAQ Interface

6.2 Student Portal Interfaces

6.2.1 Student Authentication Framework

Login & Forgot Password Screens: Dual interface flow consisting of the standard academic login portal alongside the initial email-based account recovery input screen

The image shows two mobile app screens for student authentication. The left screen, titled 'Welcome Back', has a subtitle 'Login to continue your smart learning journey.' and a 'Login' form. The form includes a 'Registration Number' field with a placeholder 'Enter registration number' and a calendar icon, a 'Password' field with a placeholder 'Enter your password' and an eye icon, a 'Remember Me' checkbox, a 'Forgot Password?' link, and a blue 'Login' button. The right screen, titled 'Reset Password', has a subtitle 'Enter your email address and we'll send you a verification OTP.' and an 'Email Verification' form. The form includes an 'Email' field with a placeholder 'mailkblabf7@gmail.com' and an envelope icon, and a blue 'Sending OTP...' button.

Figure 6.9 & 6.10: Student Login and Forgot Password Interfaces

OTP Verification & New Password Screens: Secure verification framework displaying the 4-digit verification code field side-by-side with the final password reset credentials entry form.

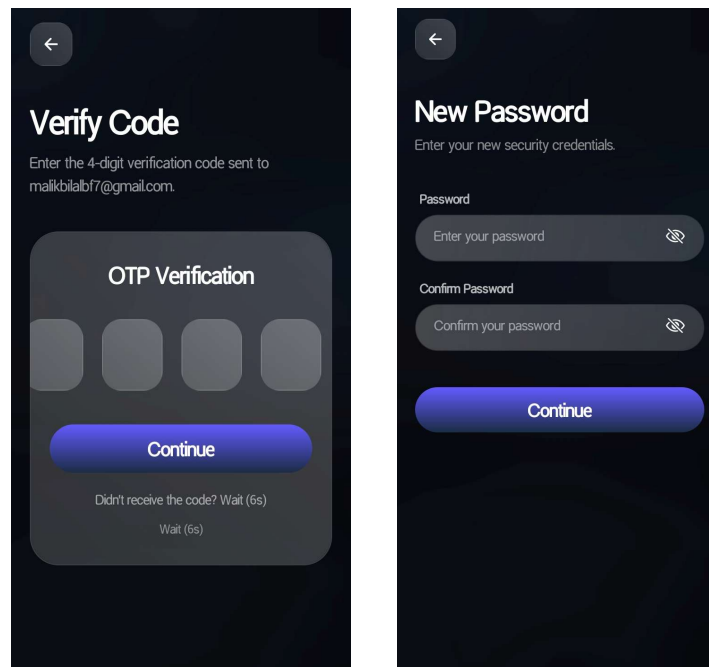
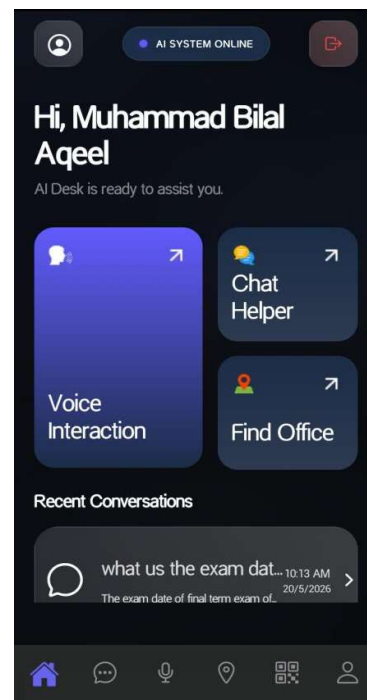


Figure 6.11 & 6.12: OTP Verification and New Password Configurations Screen

6.2.2 Student Home & Dashboard Layout

Main Home Screen: Primary navigation grid displaying feature module cards alongside a scrollable recent chat list.

Figure 6.13: Student Home Dashboard



6.2.3 Chat Module & Voice Assistant

Chat Interface with History Drawer: Main RAG chat container paired with a collapsible left drawer displaying past sessions and a "New Chat" option.

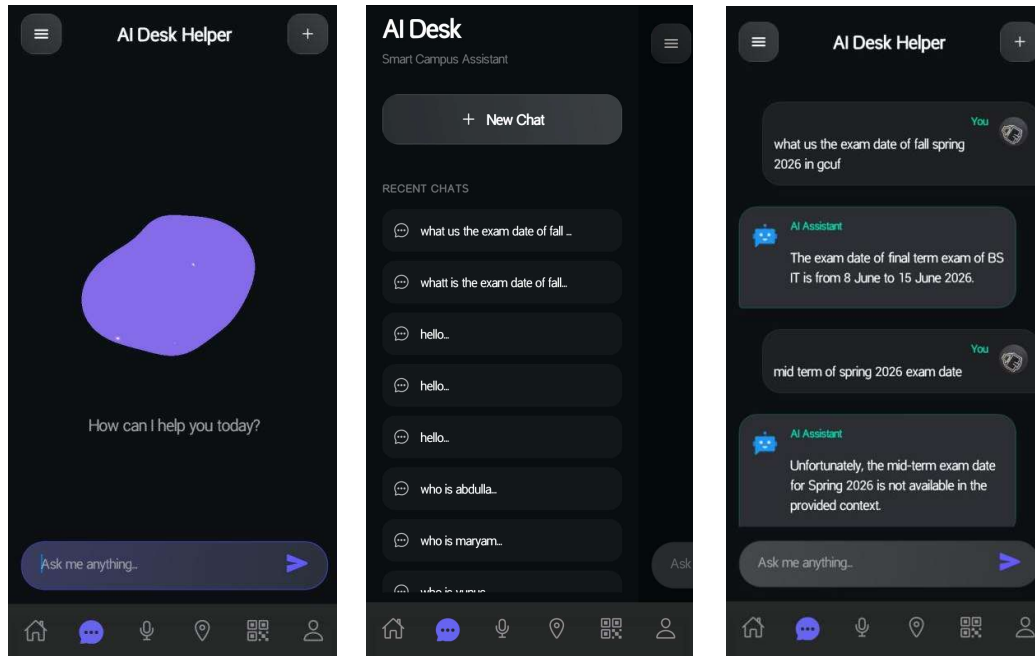


Figure 6.14, 6.15 & 6.16: AI Chat Interface with Navigation Drawer

Voice Query Page: Audio capture window leveraging the Whisper model, showing a live voice-input waveform animation.

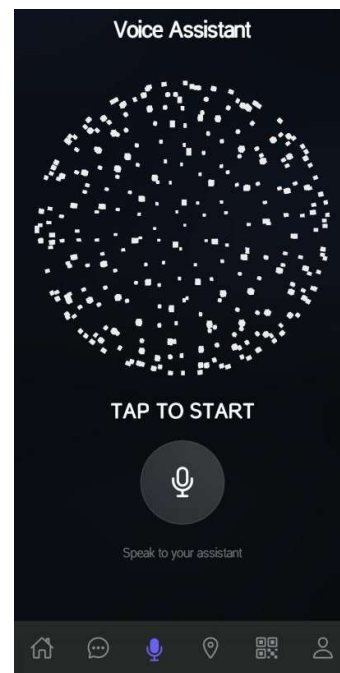
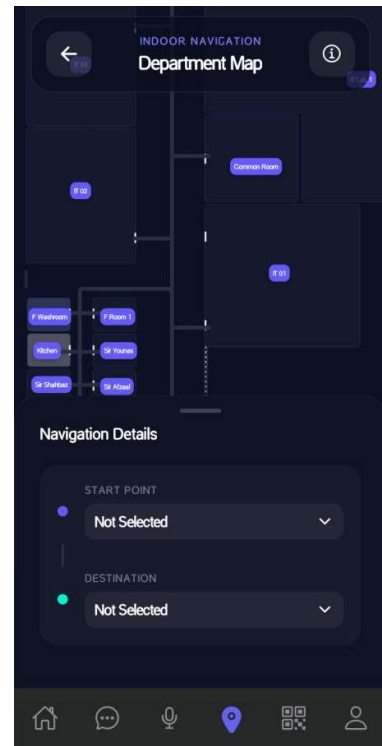


Figure 6.17: Voice Input Assistant Interface

6.2.4 Department Indoor Navigation & QR Scanner Module

Indoor Navigation Page: Interactive campus floor layout map showing spatial paths to classrooms, labs, and offices.

Figure 6.18: Indoor Navigation View



QR Scanner Page: Embedded camera scanner framework used to scan local codes outside rooms to pull up instant location logs.

Figure 6.19: Integrated QR Code Scanner



6.2.5 Profile Management

Student Profile Screen: Settings control dashboard displaying personal details, app preferences, and session logout controls.

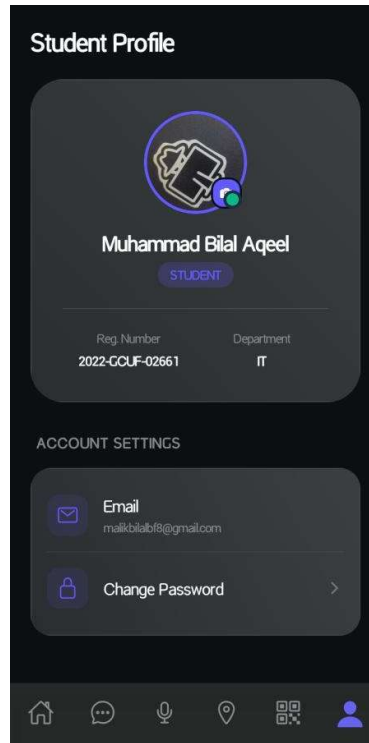


Figure 6.20: Student Profile Screen